

Guia Oficial de Engenharia Front-end — Santo Pegasus Soluciones

Localidade: São Paulo - SP

Versão: 2.0.0

Classificação: Uso Interno / Técnico

Data: 24 de Maio de 2024

1. Introdução: O papel do Front-end na Santo Pegasus

Na Santo Pegasus Soluciones, a engenharia de front-end transcende a simples construção de interfaces. Ela é o ponto de convergência entre a lógica de negócio complexa e a experiência do usuário final. Nosso objetivo é construir sistemas resilientes, escaláveis e de alta performance que reflitam a excelência técnica da nossa sede em São Paulo.

O engenheiro front-end da Santo Pegasus não é apenas um "desenvolvedor de telas", mas um arquiteto de software responsável pela integridade dos dados no cliente, pela acessibilidade e pela ponte de comunicação eficiente com nossos serviços de back-end.

1.1. O Front-end como Camada Estratégica de Negócio

Em um cenário competitivo onde a experiência digital é frequentemente o único ponto de contato entre a Santo Pegasus e seus clientes finais, o front-end assume um papel estratégico que vai muito além da entrega visual. Cada decisão técnica tomada nesta camada — seja a escolha de uma biblioteca de estado, a estratégia de cache de dados ou a forma como tratamos erros de rede — impacta diretamente métricas de negócio como taxa de conversão, retenção de usuários e percepção de qualidade da marca.

Nossos times de front-end trabalham em estreita colaboração com Product Managers, Designers de UX/UI e Engenheiros de Back-end, formando o que chamamos de "tríade de entrega". Essa colaboração multidisciplinar garante que decisões técnicas nunca sejam tomadas isoladamente, mas sempre considerando o impacto no usuário final e nos objetivos de negócio da empresa.

1.2. Desafios Técnicos Específicos

Os produtos da Santo Pegasus operam em contextos de alta criticidade — sistemas financeiros, plataformas de saúde e ferramentas administrativas de larga escala. Isso impõe desafios técnicos únicos:

- **Consistência de dados em tempo real:** Interfaces que refletem mudanças de estado quase instantaneamente, mesmo sob alta concorrência de usuários.
- **Resiliência a falhas de rede:** Aplicações que degradam graciosamente quando a conectividade é instável, sem perda de dados do usuário.

- **Escalabilidade de código:** Bases de código que crescem de forma sustentável, permitindo que múltiplos times trabalhem simultaneamente sem conflitos constantes.
- **Conformidade regulatória:** Aderência estrita a normas como LGPD, WCAG e, em alguns produtos, regulamentações específicas do setor financeiro (BACEN) e de saúde (ANVISA/CFM).

1.3. Filosofia de Engenharia

Acreditamos que código de qualidade é aquele que pode ser lido, entendido e modificado por qualquer engenheiro do time, não apenas por quem o escreveu originalmente. Por isso, priorizamos:

1. **Legibilidade sobre brevidade:** Código explícito é preferível a código "inteligente" e condensado.
2. **Testabilidade desde o design:** Componentes e hooks são desenhados pensando em como serão testados.
3. **Documentação viva:** Comentários e documentação devem explicar o "porquê", não o "o quê" — o código já expressa o "o quê".
4. **Feedback rápido:** Pipelines de CI/CD, linters e type-checking devem fornecer feedback em segundos, não em minutos.

2. Princípios de Engenharia

Adotamos princípios de design de software clássicos adaptados ao ecossistema React para garantir que o código seja sustentável a longo prazo.

2.1. SOLID em React

S — Single Responsibility Principle

Cada componente deve fazer apenas uma coisa. Se um componente gerencia estado global, faz chamadas de API e renderiza listas complexas, ele deve ser refatorado.

```
````tsx
// Violação: componente faz busca de dados, lógica de negócio e renderização
function UserProfile({ userId }: { userId: string }) {
 const [user, setUser] = useState<User | null>(null);
 const [loading, setLoading] = useState(true);
 useEffect(() => {
 fetch(`/api/users/${userId}`)
 .then((res) => res.json())
 .then((data) => {
 setUser(data);
 });
 });
}
```

```

 setLoading(false);
 });
}, [userId]);
if (loading) return Carregando...;
return <div>{user?.name}</div>;
}
// Correto: responsabilidades separadas
function useUser(userId: string) {
 return useQuery({
 queryKey: ["user", userId],
 queryFn: () => api.get<User>(`/users/${userId}`),
 });
}
function UserProfile({ userId }: { userId: string }) {
 const { data: user, isLoading } = useUser(userId);
 if (isLoading) return <Spinner />;
 return <UserProfileView user={user} />;
}
...

```

#### #### O — Open/Closed Principle

Componentes devem estar abertos para extensão (via props e composição), mas fechados para modificação interna.

```

````tsx
// Componente extensível via composição, sem alterar seu código-fonte
interface CardProps {
  children: React.ReactNode;
  className?: string;
}
function Card({ children, className }: CardProps) {
  return <div className={cn("rounded-lg border p-4", className)}>{children}</div>;
}
// Extensão via composição, sem modificar o componente Card
function UserCard({ user }: { user: User }) {
  return (
    <Card className="bg-slate-50">
      <CardHeader>{user.name}</CardHeader>
      <CardBody>{user.email}</CardBody>
    </Card>
  );
}

```

```

    </Card>
  );
}
...

```

L — Liskov Substitution Principle

Componentes que derivam de elementos base (ex: um `CustomButton` que estende `Button`) devem poder substituir seus pais sem quebrar a aplicação.

```

``tsx
type ButtonProps = React.ButtonHTMLAttributes<HTMLButtonElement>;
function Button({ className, ...props }: ButtonProps) {
  return <button className={cn("rounded-md px-4 py-2", className)} {...props} />;
}
// PrimaryButton deve ser substituível em qualquer lugar que use Button
function PrimaryButton(props: ButtonProps) {
  return <Button className="bg-pegasus-blue text-white" {...props} />;
}
...

```

I — Interface Segregation Principle

Não obrigue um componente a depender de props que ele não utiliza. Utilize TypeScript para definir contratos claros.

```

``tsx
// Interface "gorda" força consumo de props desnecessárias
interface TableProps {
  data: Row[];
  onEdit: (id: string) => void;
  onDelete: (id: string) => void;
  onExport: () => void; // Nem todo consumidor precisa de export
}
// Interfaces segregadas por responsabilidade
interface BaseTableProps {
  data: Row[];
}
interface EditableTableProps extends BaseTableProps {
  onEdit: (id: string) => void;
  onDelete: (id: string) => void;
}
interface ExportableTableProps extends BaseTableProps {

```

```

    onExport: () => void;
  }
  ...

```

D — Dependency Inversion Principle

Dependa de abstrações (hooks, contextos) em vez de implementações concretas dentro dos componentes de UI.

```

````tsx
// Componente depende diretamente da implementação do Axios
function OrderList() {
 const [orders, setOrders] = useState([]);
 useEffect(() => {
 axios.get("/api/orders").then((res) => setOrders(res.data));
 }, []);
 return {orders.map((o) => <li key={o.id}>{o.name})};
}
// Componente depende da abstração (hook), não da implementação
function useOrders() {
 return useQuery({ queryKey: ["orders"], queryFn: orderService.getAll });
}
function OrderList() {
 const { data: orders = [] } = useOrders();
 return {orders.map((o) => <li key={o.id}>{o.name})};
}
...

```

## 2.2. DRY, KISS e YAGNI

#### #### DRY (Don't Repeat Yourself)

*Abstraia lógicas repetitivas em Custom Hooks.*

```

````tsx
// Hook reutilizável para debounce, evitando duplicação em múltiplos componentes
function useDebounce<T>(value: T, delay = 300): T {
  const [debouncedValue, setDebouncedValue] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debouncedValue;
}

```

```

}
// Uso em qualquer componente de busca
function SearchInput() {
  const [term, setTerm] = useState("");
  const debouncedTerm = useDebounce(term);
  useEffect(() => {
    if (debouncedTerm) searchService.query(debouncedTerm);
  }, [debouncedTerm]);
  return <input value={term} onChange={(e) => setTerm(e.target.value)} />;
}
...

```

KISS (Keep It Simple, Stupid)

Evite "over-engineering". Se uma solução simples resolve o problema de forma legível, prefira-a.

```

````tsx
// Complexidade desnecessária para um caso simples
const isEven = (n: number) =>
 new Array(n).fill(null).reduce((acc, _, i) => (i === n - 1 ? n % 2 === 0 : acc), false);
// Solução simples e direta
const isEven = (n: number) => n % 2 === 0;
...

```

#### #### YAGNI (You Ain't Gonna Need It)

Não implemente funcionalidades ou abstrações "para o futuro" que não foram solicitadas no requisito atual.

```

````tsx
// Abstração prematura para "possíveis" múltiplos provedores de pagamento,
// quando o requisito atual só pede Pix
interface PaymentStrategy {
  process(): void;
}
class PixStrategy implements PaymentStrategy { process() {} }
class CreditCardStrategy implements PaymentStrategy { process() {} } // não solicitado ainda
class PaymentProcessor {
  constructor(private strategy: PaymentStrategy) {}
  execute() { this.strategy.process(); }
}
// Implementação direta para o requisito atual

```

```
function processPixPayment(orderId: string) {
  return api.post(`/orders/${orderId}/pix`);
}
...
---
```

3. Stack Tecnológico Padrão

Nossa stack é selecionada para oferecer o melhor equilíbrio entre produtividade do desenvolvedor e performance para o usuário.

- **React 18 (mínimo 18.2.0):** Utilizando as novas APIs de concorrência e Transitions.
- **TypeScript (mínimo 5.0):** Uso obrigatório de tipagem estrita (``strict: true``) para reduzir erros em tempo de execução.
- **Next.js (mínimo 14.x, App Router):** Framework principal para aplicações voltadas ao público (SEO e Performance via SSR/SSG).
- **Vite (mínimo 5.x):** Ferramenta de build para SPAs (Single Page Applications) internas e dashboards de baixa necessidade de SEO.
- **Node.js (mínimo LTS 20.x):** Runtime padrão para build e ferramentas de desenvolvimento.

3.1. Critérios de Decisão: Next.js vs Vite

A escolha entre os dois frameworks não é arbitrária. Utilizamos a seguinte matriz de decisão:

Critério	Next.js	Vite (SPA)
Necessidade de SEO / indexação	Obrigatório	Não aplicável
Renderização no servidor (SSR/SSG)	Sim, nativo	Não (apenas client-side)
Aplicação voltada ao público externo	Sim	Raramente
Dashboard interno / ferramenta administrativa	Possível, mas não ideal	Recomendado
Tempo de build e complexidade de infra	Maior (exige runtime Node)	Menor (build estático)
Roteamento baseado em arquivos	Sim (App Router)	Requer biblioteca (React Router)

Regra prática: se o produto será acessado por usuários anônimos via mecanismos de busca (marketing, landing pages, portal do cliente), utilize Next.js. Se o produto é uma ferramenta interna, autenticada, sem necessidade de indexação (ex: painel administrativo, dashboard de operações), utilize Vite.


```

useDebounce.ts
useMediaQuery.ts
services/          # Camada de comunicação com API
  api.ts           # Instância Axios
  userService.ts
  orderService.ts
stores/            # Zustand stores
  useAuthStore.ts
  useThemeStore.ts
types/             # Tipos globais e gerados via OpenAPI
  api-generated.d.ts
  domain.ts
lib/               # Utilitários (cn, formatters, etc.)
  utils.ts
styles/
  globals.css
tests/
  e2e/
    checkout.spec.ts  # Playwright
...

```

4.3. Separação de Lógica e UI

Adotamos o padrão de Container/Presenter via Hooks:

- A lógica de estado e chamadas de API residem em um Hook customizado (ex: `useUserDashboard.ts`).
- O componente React recebe os dados e funções via props, focando apenas na renderização.

Exemplo completo — `useUserDashboard.ts` (Container):

```

````tsx
// hooks/useUserDashboard.ts
import { useQuery } from "@tanstack/react-query";
import { useMemo, useState } from "react";
import { orderService } from "@services/orderService";
import { userService } from "@services/userService";
interface UseUserDashboardParams {
 userId: string;
}

```

```

export function useUserDashboard({ userId }: UseUserDashboardParams) {
 const [statusFilter, setStatusFilter] = useState<"all" | "pending" | "done">("all");
 const userQuery = useQuery({
 queryKey: ["user", userId],
 queryFn: () => userService.getByld(userId),
 });
 const ordersQuery = useQuery({
 queryKey: ["orders", userId, statusFilter],
 queryFn: () => orderService.getByUser(userId, statusFilter),
 enabled: !!userQuery.data,
 });
 const totalSpent = useMemo(
 () => ordersQuery.data?.reduce((sum, order) => sum + order.total, 0) ?? 0,
 [ordersQuery.data]
);
 return {
 user: userQuery.data,
 orders: ordersQuery.data ?? [],
 totalSpent,
 isLoading: userQuery.isLoading || ordersQuery.isLoading,
 isError: userQuery.isError || ordersQuery.isError,
 statusFilter,
 setStatusFilter,
 };
}
...

```

### Exemplo completo — `UserDashboard.tsx` (Presenter):

```

```tsx
// components/organisms/UserDashboard/UserDashboard.tsx
import { useUserDashboard } from "../useUserDashboard";
import { Spinner } from "@components/ui/spinner";
import { OrderList } from "@components/molecules/OrderList";
interface UserDashboardProps {
  userId: string;
}
export function UserDashboard({ userId }: UserDashboardProps) {
  const { user, orders, totalSpent, isLoading, isError, statusFilter, setStatusFilter } =

```

```

    useUserDashboard({ userId });
    if (isLoading) return <Spinner aria-label="Carregando painel do usuário" />;
    if (isError) return <p role="alert">Erro ao carregar dados do usuário.</p>;
    return (
      <section aria-labelledby="dashboard-title">
        <h1 id="dashboard-title">Olá, {user?.name}</h1>
        <p>Total gasto: R$ {totalSpent.toFixed(2)}</p>
        <OrderList orders={orders} statusFilter={statusFilter} onFilterChange={setStatusFilter} />
      </section>
    );
  }
  ...
  ---

```

5. Gerenciamento de Estado

Diferenciamos rigidamente o Estado do Servidor do Estado da UI.

- **Zustand:** Utilizado para estados globais da aplicação (ex: dados do usuário logado, preferências de tema, estado do carrinho). É leve e evita o boilerplate do Redux.
- **React Query (TanStack Query):** Utilizado para sincronização de dados com o back-end. Gerencia cache, estados de loading, error e re-fetch de forma automática.

5.1. Exemplo de Store Zustand

```

````tsx
// stores/useAuthStore.ts
import { create } from "zustand";
import { persist } from "zustand/middleware";
interface AuthState {
 user: { id: string; name: string; role: string } | null;
 isAuthenticated: boolean;
 setUser: (user: AuthState["user"]) => void;
 logout: () => void;
}
export const useAuthStore = create<AuthState>()(
 persist(
 (set) => ({
 user: null,
 isAuthenticated: false,
 setUser: (user) => set({ user, isAuthenticated: !!user }),

```

```

 logout: () => set({ user: null, isAuthenticated: false }),
 }),
 { name: "pegasus-auth-storage" }
)
);
...

```

## 5.2. Exemplo de Hook React Query

```

``tsx
// services/orderService.ts
import { api } from "../api";
import type { Order } from "@types/domain";
export const orderService = {
 getByUser: (userId: string, status: string) =>
 api.get<Order[]>(`/users/${userId}/orders`, { params: { status } }).then((r) => r.data),
};
// hooks/useOrders.ts
import { useQuery } from "@tanstack/react-query";
import { orderService } from "@services/orderService";
export function useOrders(userId: string, status: string) {
 return useQuery({
 queryKey: ["orders", userId, status],
 queryFn: () => orderService.getByUser(userId, status),
 staleTime: 60 * 1000, // 1 minuto de cache "fresco"
 retry: 2,
 });
}
...

```

## 6. Consumo de APIs e Segurança de Tokens

### 6.1. Axios e Interceptores

Utilizamos instâncias centralizadas do Axios. Interceptores são obrigatórios para:

- Injetar cabeçalhos de autenticação.
- Tratamento global de erros (ex: redirecionar para `/login` em caso de erro 401).
- Logging de performance em ambiente de staging.

**Exemplo completo de configuração:**

```

``tsx
// services/api.ts
import axios, { AxiosError } from "axios";
import { useAuthStore } from "@stores/useAuthStore";
export const api = axios.create({
 baseURL: process.env.NEXT_PUBLIC_API_URL,
 withCredentials: true, // envio de cookies HttpOnly
 timeout: 10_000,
});
// Interceptor de REQUEST
api.interceptors.request.use((config) => {
 const startTime = performance.now();
 config.headers["X-Request-Start"] = String(startTime);
 if (process.env.NODE_ENV === "staging") {
 console.info(`[API] → ${config.method?.toUpperCase()} ${config.url}`);
 }
 return config;
});
// Interceptor de RESPONSE
api.interceptors.response.use(
 (response) => {
 if (process.env.NODE_ENV === "staging") {
 const start = Number(response.config.headers["X-Request-Start"]);
 console.info(`[API] ← ${response.config.url} (${(performance.now() - start).toFixed(0)}ms)`);
 }
 return response;
 },
 (error: AxiosError) => {
 if (error.response?.status === 401) {
 useAuthStore.getState().logout();
 window.location.href = "/login";
 }
 if (error.response?.status === 403) {
 console.error("Acesso negado ao recurso solicitado.");
 }
 return Promise.reject(error);
 }
}

```

```
);
...
```

## 6.2. Autenticação JWT

Na Santo Pegasus, a segurança é prioridade.

- **Proibido:** Armazenar JWT no ``localStorage`` ou ``sessionStorage`` devido a ataques XSS.
- **Padrão:** O token deve ser trafegado via Cookies `HttpOnly`, configurados pelo back-end. O front-end apenas envia as credenciais com ``withCredentials: true``.

---

## 7. Tipagem de API com TypeScript

### 7.1. Interfaces e Tipos Utilitários

Toda comunicação com o back-end deve ser tipada, garantindo que erros de contrato sejam detectados em tempo de compilação.

```
```tsx  
// types/domain.ts  
export interface Order {  
  id: string;  
  userId: string;  
  status: "pending" | "processing" | "done" | "cancelled";  
  total: number;  
  createdAt: string;  
}  
  
// Tipos utilitários do TypeScript aplicados ao domínio  
export type CreateOrderPayload = Omit<Order, "id" | "createdAt">;  
export type OrderSummary = Pick<Order, "id" | "status" | "total">;  
export type PartialOrderUpdate = Partial<Pick<Order, "status">>;  
```
```

### 7.2. Geração Automática de Tipos a partir do OpenAPI/Swagger

Para evitar divergências manuais entre front-end e back-end, utilizamos geração automática de tipos a partir do contrato Swagger exposto pelos microsserviços.

```
```bash
```

Ferramenta padrão: openapi-typescript

```
npx openapi-typescript https://api.pegasus.com/swagger.json \  
  --output src/types/api-generated.d.ts  
```
```

```

````tsx
// Uso dos tipos gerados automaticamente
import type { components } from "@types/api-generated";
type ApiOrder = components["schemas"]["Order"];
const orderService = {
  getById: (id: string) => api.get<ApiOrder>(`/orders/${id}`).then((r) => r.data),
};
````

```

Esse comando é executado como parte do pipeline de CI, garantindo que qualquer mudança de contrato no back-end seja refletida automaticamente nos tipos do front-end, com falha de build caso haja quebra de compatibilidade.

---

## 8. Estilização e Design Tokens

### 8.1. Tailwind CSS + shadcn/ui

- **Tailwind CSS:** Para estilização rápida baseada em utilitários, garantindo consistência visual.
- **shadcn/ui:** Nossa biblioteca de componentes base. Os componentes são copiados para dentro do projeto (`components/ui`) para permitir total customização do código-fonte.

### 8.2. Design Tokens no `tailwind.config.js`

Valores como cores corporativas, espaçamentos e raios de borda devem ser definidos no arquivo `tailwind.config.js` para garantir que mudanças de marca sejam aplicadas globalmente de forma instantânea.

```

````js
// tailwind.config.js
/* @type {import('tailwindcss').Config} */
module.exports = {
  content: ["/src/**/*.ts,tsx"],
  theme: {
    extend: {
      colors: {
        "pegasus-blue": {
          50: "#eef4ff",
          500: "#2563eb",
          700: "#1d4ed8",
          900: "#1e3a8a",

```

```

    },
    "pegasus-gray": {
      100: "#f3f4f6",
      800: "#1f2937",
    },
  },
  spacing: {
    "18": "4.5rem",
    "22": "5.5rem",
  },
  borderRadius: {
    pegasus: "0.625rem",
  },
  fontFamily: {
    sans: ["Inter", "system-ui", "sans-serif"],
  },
},
},
},
plugins: [require("tailwindcss-animate")],
};
...

```

9. Gerenciamento de Formulários com React Hook Form + Zod

Formulários são pontos críticos de captura de dados e devem seguir um padrão único de validação e submissão.

9.1. Schema de Validação com Zod

```

````tsx
// schemas/orderFormSchema.ts
import { z } from "zod";
export const orderFormSchema = z.object({
 customerName: z.string().min(3, "Nome deve ter ao menos 3 caracteres"),
 email: z.string().email("E-mail inválido"),
 quantity: z.number().int().positive("Quantidade deve ser maior que zero"),
 paymentMethod: z.enum(["pix", "credit_card", "boleto"], {
 errorMap: () => ({ message: "Selecione um método de pagamento válido" }),
 }),
});

```

```
});
export type OrderFormData = z.infer<typeof orderFormSchema>;
...
```

## 9.2. Integração com React Hook Form

```
``tsx
// components/organisms/OrderForm/OrderForm.tsx
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { orderFormSchema, type OrderFormData } from "@schemas/orderFormSchema";
export function OrderForm({ onSubmit }: { onSubmit: (data: OrderFormData) => void }) {
 const {
 register,
 handleSubmit,
 formState: { errors, isSubmitting },
 } = useForm<OrderFormData>({
 resolver: zodResolver(orderFormSchema),
 });
 return (
 <form onSubmit={handleSubmit(onSubmit)} noValidate>
 <label htmlFor="customerName">Nome</label>
 <input id="customerName" {...register("customerName")}
 aria-invalid={!!errors.customerName} />

 {errors.customerName && {errors.customerName.message}}
 <label htmlFor="email">E-mail</label>
 <input id="email" type="email" {...register("email")} aria-invalid={!!errors.email} />
 {errors.email && {errors.email.message}}
 <button type="submit" disabled={isSubmitting}>
 {isSubmitting ? "Enviando..." : "Confirmar pedido"}
 </button>
 </form>
);
}

```

## 10. Acessibilidade (a11y)

Seguimos a norma WCAG 2.1 nível AA.

## 10.1. Checklist Detalhado WCAG 2.1 AA

- [ ] Perceptível: Contraste mínimo de 4.5:1 para texto normal e 3:1 para texto grande.
- [ ] Perceptível: Toda imagem informativa possui `alt` descritivo; imagens decorativas usam `alt=""`.
- [ ] Operável: Todos os elementos interativos são acessíveis via `Tab` e possuem foco visível (`:focus-visible`).
- [ ] Operável: Nenhuma armadilha de foco (focus trap) não intencional em modais.
- [ ] Compreensível: Mensagens de erro em formulários são anunciadas via `aria-live` ou `role="alert"`.
- [ ] Compreensível: Idioma da página declarado corretamente (`<html lang="pt-BR">`).
- [ ] Robusto: HTML validado sem elementos aninhados incorretamente.
- [ ] Uso correto de HTML semântico (`<main>`, `<nav>`, `<section>`).
- [ ] Atributos `aria-label` em elementos iconográficos.
- [ ] Suporte total à navegação via teclado (foco visível).
- [ ] Contraste de cores validado por ferramentas automatizadas (axe-core, Lighthouse).

## 10.2. Exemplos de Código com ARIA

```
``tsx
// Botão apenas com ícone precisa de aria-label descritivo
<button aria-label="Remover item do carrinho" onClick={handleRemove}>
 <TrashIcon aria-hidden="true" />
</button>

// Região de status dinâmico anunciada por leitores de tela
<div role="status" aria-live="polite">
 {isLoading ? "Carregando resultados..." : `${results.length} resultados encontrados`}
</div>

// Modal com foco gerenciado e rótulo acessível
<div role="dialog" aria-modal="true" aria-labelledby="modal-title">
 <h2 id="modal-title">Confirmar exclusão</h2>
 <button onClick={onClose} aria-label="Fechar modal">x</button>
</div>
...

```

## 11. Testes Automatizados

Nossa meta de cobertura de código é de 80%.

1. Vitest: Runner de testes unitários para funções puras e hooks.

**2. React Testing Library (RTL):** Para testes de integração de componentes, focando no comportamento do usuário e não na implementação interna.

**3. Playwright:** Para testes E2E (End-to-End) em fluxos críticos, como o checkout e o login.

### 11.1. Exemplo de Teste Unitário com Vitest

```
````tsx
// hooks/useDebounce.test.ts
import { describe, it, expect, vi } from "vitest";
import { renderHook, act } from "@testing-library/react";
import { useDebounce } from "../useDebounce";
describe("useDebounce", () => {
  it("deve atualizar o valor apenas após o delay", () => {
    vi.useFakeTimers();
    const { result, rerender } = renderHook(({ value }) => useDebounce(value, 300), {
      initialProps: { value: "a" },
    });
    rerender({ value: "ab" });
    expect(result.current).toBe("a"); // ainda não atualizou
    act(() => vi.advanceTimersByTime(300));
    expect(result.current).toBe("ab");
    vi.useRealTimers();
  });
});
````
```

### 11.2. Exemplo de Teste de Componente com Testing Library

```
````tsx
// components/organisms/OrderForm/OrderForm.test.tsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { OrderForm } from "../OrderForm";
describe("OrderForm", () => {
  it("exibe erro de validação quando e-mail é inválido", async () => {
    const onSubmit = vi.fn();
    render(<OrderForm onSubmit={onSubmit} />);
    await userEvent.type(screen.getByLabelText("Nome"), "Ana");
    await userEvent.type(screen.getByLabelText("E-mail"), "email-invalido");
    await userEvent.click(screen.getByRole("button", { name: /confirmar pedido/i }));
  });
});
````
```

```

 expect(await screen.findByRole("alert")).toHaveTextContent("E-mail inválido");
 expect(onSubmit).not.toHaveBeenCalled();
 });
});
...

```

### 11.3. Testes E2E com Playwright

Testes E2E validam fluxos críticos de ponta a ponta, simulando o comportamento real do usuário em um navegador.

```

````tsx
// tests/e2e/checkout.spec.ts
import { test, expect } from "@playwright/test";
test.describe("Fluxo de Checkout", () => {
  test("usuário completa uma compra com sucesso", async ({ page }) => {
    await page.goto("/produtos/123");
    await page.getByRole("button", { name: "Adicionar ao carrinho" }).click();
    await page.getByRole("link", { name: "Ver carrinho" }).click();
    await page.getByRole("button", { name: "Finalizar compra" }).click();
    await page.getByLabel("Nome completo").fill("Maria Silva");
    await page.getByLabel("E-mail").fill("maria@pegasus.com");
    await page.getByLabel("Método de pagamento").selectOption("pix");
    await page.getByRole("button", { name: "Confirmar pedido" }).click();
    await expect(page.getByText("Pedido confirmado com sucesso")).toBeVisible();
    await expect(page).toHaveURL(/\/pedidos\/\d+\/confirmacao/);
  });
});
...

```

12. GitFlow e Padrão de Commits

Seguimos o Conventional Commits para manter um histórico de mensagens legível e permitir a geração automática de changelogs.

Formato: `tipo(escopo): descrição curta`

- `feat`: Nova funcionalidade.
- `fix`: Correção de bug.
- `chore`: Manutenção de build, dependências, etc.
- `docs`: Mudanças na documentação.

13. Performance e Core Web Vitals

Aplicações da Santo Pegasus devem ser otimizadas para os indicadores do Google:

- **LCP (Largest Contentful Paint):** Otimização de imagens (Next/Image) e prioridade de carregamento.
- **FID (First Input Delay):** Minimização do tempo de execução do JavaScript no fio principal (Main Thread).
- **CLS (Cumulative Layout Shift):** Reserva de espaço para elementos carregados assincronamente para evitar pulos na interface.

13.1. Lazy Loading e Code Splitting

```
``tsx
// Carregamento assíncrono de componentes pesados
import { lazy, Suspense } from "react";
const ReportChart = lazy(() => import("@/components/organisms/ReportChart"));
function ReportsPage() {
  return (
    <Suspense fallback={<Spinner aria-label="Carregando gráfico" />}>
      <ReportChart />
    </Suspense>
  );
}
...

``tsx
// Divisão de rotas em Next.js — code splitting automático por rota
// app/relatorios/page.tsx é servido como bundle independente do restante da aplicação
...
```

13.2. Otimização de Imagens com Next/Image

```
``tsx
import Image from "next/image";
function ProductBanner() {
  return (
    <Image
      src="/banners/promocao.jpg"
      alt="Promoção de produtos Pegasus"
      width={1200}
      height={400}
    />
  );
}
```

```

priority // usada para imagens acima da dobra (impacta LCP)
placeholder="blur"
blurDataURL="/banners/promocao-blur.jpg"
/>
);
}
...
---
```

14. Segurança e LGPD

14.1. Proteção no Navegador

- **XSS (Cross-Site Scripting):** Sanitização de qualquer entrada de usuário antes da renderização. Nunca utilize `dangerouslySetInnerHTML` sem validação rigorosa.
- **CSRF:** Proteção via tokens de sincronização e política de `SameSite` nos cookies.

14.2. Exemplo Prático de Sanitização

```

````tsx
import DOMPurify from "dompurify";
// Quando é estritamente necessário renderizar HTML dinâmico (ex: editor de texto rico)
function CommentContent({ rawHtml }: { rawHtml: string }) {
 const sanitizedHtml = DOMPurify.sanitize(rawHtml, {
 ALLOWED_TAGS: ["b", "i", "em", "strong", "p"],
 ALLOWED_ATTR: [],
 });
 return <div dangerouslySetInnerHTML={{ __html: sanitizedHtml }} />;
}
// Preferência sempre por renderização segura via texto puro
function SafeComment({ text }: { text: string }) {
 return <p>{text}</p>; // React já escapa automaticamente
}
...
````
```

14.3. Conformidade com a LGPD

Como operamos em São Paulo, o front-end deve refletir a transparência exigida pela Lei Geral de Proteção de Dados:

- **Gestão de Consentimento:** Banners de cookies que permitem ao usuário optar por rastreamento analítico ou marketing.

- **Minimização de Dados:** Não coletar ou armazenar no estado do front-end dados sensíveis que não sejam estritamente necessários para a execução da tarefa atual.
- **Direito de Exclusão:** Interfaces claras para que o usuário solicite a exclusão de seus dados, integradas aos nossos endpoints de privacidade.

15. CI/CD para Front-end

15.1. Pipeline Padrão (GitHub Actions)

```yaml

**.github/workflows/frontend-ci.yml**

name: Frontend CI/CD

on:

pull\_request:

branches: [main, develop]

push:

branches: [main]

jobs:

lint-test-build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with:

node-version: "20"

cache: "npm"

- run: npm ci

- run: npm run lint

- run: npm run type-check

- run: npm run test -- --coverage

- run: npm run build

e2e:

needs: lint-test-build

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- run: npx playwright install --with-deps

```

- run: npm run test:e2e
docker-deploy:
 needs: [lint-test-build, e2e]
 if: github.ref == 'refs/heads/main'
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - run: docker build -t pegasus-frontend:${{ github.sha }} .
 - run: docker push registry.pegasus.com/frontend:${{ github.sha }}
...

```

## 15.2. Dockerfile de Referência

```

```dockerfile

```

Dockerfile

```

FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json ./
RUN npm ci
COPY . .
RUN npm run build
FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next ./next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package.json ./
RUN npm ci --omit=dev
EXPOSE 3000
CMD ["npm", "start"]
...

```

16. Monitoramento de Erros em Produção

Utilizamos Sentry como ferramenta padrão de rastreamento de erros e performance no front-end.

```

```tsx
// lib/sentry.ts
import { Sentry } from "@sentry/nextjs";

```

```

Sentry.init({
 dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
 tracesSampleRate: 0.2,
 environment: process.env.NODE_ENV,
 beforeSend(event) {
 // Nunca enviar dados sensíveis (LGPD) para o Sentry
 delete event.user?.email;
 return event;
 },
});
...

````tsx
// Captura manual de erros em pontos críticos
try {
  await orderService.create(payload);
} catch (error) {
  Sentry.captureException(error, { tags: { feature: "checkout" } });
  throw error;
}
...

```

Alertas são configurados para notificar o time responsável via Slack sempre que a taxa de erros de uma release exceder o limiar de 1% das sessões ativas.

17. Glossário de Termos Técnicos

- **SSR (Server-Side Rendering):** Renderização do HTML no servidor a cada requisição.
- **SSG (Static Site Generation):** Geração do HTML em tempo de build, servido como arquivo estático.
- **Hydration:** Processo de "ativação" do JavaScript React sobre o HTML já renderizado.
- **Code Splitting:** Divisão do bundle JavaScript em partes menores, carregadas sob demanda.
- **Custom Hook:** Função JavaScript que utiliza hooks do React para encapsular lógica reutilizável.
- **Container/Presenter:** Padrão que separa lógica de estado (Container) da renderização visual (Presenter).
- **Stale Time:** Tempo em que um dado em cache é considerado "fresco" pelo React Query.

- **Bounded Context:** Fronteira lógica de um domínio de negócio (termo herdado do Domain-Driven Design).
- **XSS (Cross-Site Scripting):** Vulnerabilidade que permite injeção de scripts maliciosos em páginas.
- **CSRF (Cross-Site Request Forgery):** Ataque que induz o usuário a executar ações não intencionais em um site autenticado.
- **WCAG: Web Content Accessibility Guidelines** — diretrizes internacionais de acessibilidade web.
- **LCP, FID, CLS:** Métricas do Core Web Vitals do Google relacionadas a performance percebida.
- **Zod:** Biblioteca de validação e inferência de esquemas em TypeScript.
- **OpenAPI/Swagger:** Especificação padrão para documentação e contrato de APIs REST.

Aprovado por:

Departamento de Engenharia de Software — Santo Pegasus Soluciones

Diretoria de Tecnologia (CTO)

Guia Oficial de Engenharia Back-end — Santo Pegasus Soluciones

Versão do Documento: 3.0.0 (Edição Global de Engenharia)

Última Atualização: Outubro de 2025

Departamento: Engenharia de Software / Centro de Excelência Técnica

Idioma: Português Brasileiro (PT-BR)

1. Introdução Profunda: A Gênese da Excelência

1.1. Nossa História e Missão Tecnológica

A Santo Pegasus Soluciones nasceu da necessidade de convergir dois setores críticos da economia brasileira: o ecossistema financeiro (FinTech) e a infraestrutura de saúde (HealthTech). Desde a nossa fundação, entendemos que o software não é apenas um suporte, mas o núcleo de confiança entre a instituição e o cidadão.

Nossa missão tecnológica é "Construir sistemas resilientes que protejam a integridade dos dados e garantam a disponibilidade absoluta de serviços vitais". Para a Santo Pegasus, um milissegundo de latência em um processamento de Pix ou uma falha de sincronização em um prontuário médico não são apenas erros técnicos; são impactos diretos na vida e no patrimônio de nossos usuários.

1.2. Cultura "Engineering Excellence"

A cultura de excelência em engenharia (Engineering Excellence) na Santo Pegasus baseia-se em quatro pilares inegociáveis:

- Propriedade Técnica (Ownership):** O desenvolvedor é responsável pelo ciclo de vida completo de seu código, do design ao monitoramento em produção.
- Simplicidade Radical:** Resolver problemas complexos com o mínimo de código necessário. Código que não existe não possui bugs.
- Segurança por Design:** A conformidade com a LGPD e a proteção contra ameaças cibernéticas começam na primeira linha de código, não no deploy.
- Evolução Contínua:** Adotamos o "Kaizen" — melhoria constante. O Guia de Engenharia é um organismo vivo que reflete o estado da arte da tecnologia.

2. Princípios de Engenharia e Design de Software (SOLID)

Na Santo Pegasus, os princípios SOLID não são sugestões; são o fundamento de nossa arquitetura. Abaixo, detalhamos cada princípio com implementações modernas em Java 17/21.

2.1. S — Single Responsibility Principle (SRP)

Definição: Uma classe deve ter uma, e apenas uma, razão para mudar.

Cenário de Refatoração:

Legado (Incorreto): Um `PaymentService` que valida o usuário, calcula taxas, salva no banco e envia e-mail.

Clean Code (Correto):

```
```java
// Responsabilidade Única: Apenas orquestra o fluxo de pagamento
@Service
@RequiredArgsConstructor
public class PaymentService {
 private final PaymentValidator validator;
 private final FeeCalculator feeCalculator;
 private final PaymentRepository repository;
 private final NotificationService notificationService;
 @Transactional
 public PaymentResponse process(PaymentRequest request) {
 validator.validate(request);
 var fee = feeCalculator.calculate(request.amount());
 var entity = repository.save(new PaymentEntity(request, fee));
 notificationService.sendReceipt(entity);
 return PaymentResponse.from(entity);
 }
}
```

## 2.2. O — Open/Closed Principle (OCP)

**Definição:** Objetos devem estar abertos para extensão, mas fechados para modificação.

**Aplicação com Java 17 Records e Sealed Interfaces:**

```
```java
public sealed interface PaymentMethod permits Pix, Boleto, CreditCard {}
public record Pix(String key) implements PaymentMethod {}
public record Boleto(String barcode) implements PaymentMethod {}
@Service
public class PaymentProcessor {
    private final Map<Class<? extends PaymentMethod>, PaymentStrategy> strategies;
    public void process(PaymentMethod method) {
```

```
        strategies.get(method.getClass()).execute(method);
    }
}
...

```

Isso permite adicionar novos métodos de pagamento sem tocar na classe `PaymentProcessor`.

2.3. L — Liskov Substitution Principle (LSP)

Definição: Subclasses devem ser substituíveis por suas classes base sem alterar a correção do programa.

Na Santo Pegasus, garantimos que nossas implementações de repositório (SQL vs NoSQL) sigam a mesma interface de contrato, assegurando que o domínio não sofra com a troca de tecnologia de persistência.

2.4. I — Interface Segregation Principle (ISP)

Definição: Muitas interfaces específicas são melhores do que uma interface geral.

Evitamos interfaces "Gordas". Em vez de `IUserService`, temos `IUserAuthenticator`, `IUserProfileManager` e `IUserDeletor`.

2.5. D — Dependency Inversion Principle (DIP)

Definição: Dependenda de abstrações, não de implementações concretas.

Utilizamos a Injeção de Dependência do Spring para garantir que nossas classes de `Service` dependam apenas de `Interfaces` de repositórios ou clientes externos.

3. Ecossistema Spring Boot 3.x e Java Moderno

3.1. Java 17/21: O Novo Padrão

- **Records:** Substituem classes POJO/Lombok para DTOs. São imutáveis por padrão e reduzem o boilerplate.
- **Sealed Classes:** Utilizadas para representar hierarquias de domínio fechadas (ex: Status de Pedido).
- **Pattern Matching for Switch:** Melhora a legibilidade em fluxos de decisão complexos.
- **Virtual Threads (Java 21):** Para serviços de alto throughput, permitindo escalar a execução concorrente sem sobrecarregar o SO.

3.2. Spring Boot Actuator e Observabilidade

Todo microserviço Santo Pegasus deve expor os endpoints de saúde e métricas:

- `/actuator/health`: Monitoramento de prontidão (Liveness/Readiness).
- `/actuator/prometheus`: Exportação de métricas para o Grafana.

- **Custom Metrics:** Utilizamos `MeterRegistry` do Micrometer para medir o volume financeiro transacionado em tempo real (R\$).

4. Arquitetura em Camadas e Hexagonal

Nossa arquitetura visa o desacoplamento total do Framework.

4.1. As Camadas

1. **Domínio (Domain):** Onde reside a inteligência do negócio. Classes PURE JAVA. Sem anotações do Spring ou JPA. Aqui definimos as `Entities` e as `Domain Rules`.
2. **Aplicação (Application):** Contém os `Use Cases`. Orquestra a lógica de negócio chamando as `Ports` (Interfaces).
3. **Infraestrutura (Infrastructure):** Camada de detalhes. Implementações de `Adapters` (Repositories, WebClients, Mensageria).

4.2. Regra de Dependência

As dependências devem sempre apontar para o Domínio. O Domínio nunca conhece o Banco de Dados ou a API REST. Isso é garantido através do padrão Ports and Adapters.

5. Estratégia RAG (IA) Avançada

Para o processamento de dados não estruturados na Santo Pegasus, implementamos pipelines de RAG (Retrieval-Augmented Generation).

5.1. Pipeline de Dados

1. **Semantic Chunking:** Em vez de divisões fixas, utilizamos análise de parágrafos para manter o contexto semântico.
2. **Embedding Model:** Padronizamos o `text-embedding-004` via AWS Bedrock ou OpenAI.
3. **Vector Store:** Uso de Qdrant em clusters de alta disponibilidade.
4. **Hybrid Search:** Combinamos busca vetorial (similaridade) com busca de texto completo (BM25) para aumentar a precisão em termos técnicos de saúde.

5.2. Prompt Engineering e Segurança

- **System Prompts:** Devem incluir restrições estritas de "Hallucination Check".
- **PII Filtering:** Antes de enviar qualquer dado para o LLM, o conteúdo passa por uma camada de anonimização para proteger dados de pacientes sob a LGPD.

6. Segurança e LGPD de Ponta a Ponta

6.1. Autenticação e Autorização (OAuth2/JWT)

- Utilizamos Keycloak ou AWS Cognito como Identity Provider (IdP).
- JWT Claims: Devem conter `scopes` específicos e `roles`. Nunca incluir senhas ou dados sensíveis no Payload do JWT.

6.2. Proteção de Dados (PII) e Criptografia

- Em Trânsito: TLS 1.3 obrigatório em todas as comunicações.
- Em Repouso: Criptografia de colunas sensíveis (CPF, Valor de Saldo) no PostgreSQL utilizando `pgcrypto` ou chaves gerenciadas no AWS KMS.
- Anonimização: Em ambientes de Homologação, os dados de produção devem ser obrigatoriamente "mascarados" ou "anonimizados".

7. Comunicação e Resiliência

7.1. Protocolos

- REST/JSON: Para integrações externas e Front-end.
- gRPC: Para comunicação inter-serviços de alta performance, utilizando Protobuf para reduzir o overhead de rede.

7.2. Padrões de Estabilidade (Resilience4j)

- Circuit Breaker: Se um serviço externo falhar, o circuito abre para evitar o efeito cascata.
- Rate Limiting: Proteção contra ataques de negação de serviço e controle de custos de API.
- Retry: Implementado com `Exponential Backoff` para falhas transitórias.

8. Banco de Dados e Migrations

8.1. Evolução de Esquema com Flyway

Toda alteração em banco de dados deve ser versionada através de scripts SQL no diretório `src/main/resources/db/migration`.

- Naming Convention: `V{VERSAO}\_\_{DESCRICAO}.sql`.

8.2. Performance JPA/Hibernate

- N+1 Problem: Uso obrigatório de `EntityGraph` ou `JOIN FETCH` em queries complexas.
- ReadOnly Transactions: Métodos de consulta devem ser anotados com `@Transactional(readOnly = true)` para otimizar o uso do Pool de conexões.

9. Estratégia de Testes e Qualidade

9.1. A Pirâmide de Testes Santo Pegasus

- 1. Testes Unitários (70%):** Testam a lógica pura dos Services e Entities. Mockamos todas as dependências com Mockito.
- 2. Testes de Integração (20%):** Validam a integração com o Banco de Dados e APIs externas. Utilizamos Testcontainers para subir instâncias reais de PostgreSQL ou Redis durante os testes.
- 3. Testes E2E/Contrato (10%):** Validam o fluxo completo. Utilizamos Pact.io para garantir que mudanças no Back-end não quebrem o Front-end ou outros serviços.

10. Infraestrutura e CI/CD

10.1. Dockerização Otimizada

Nossos Dockerfiles utilizam Multi-stage builds para garantir imagens leves e seguras:

```dockerfile

#### Stage 1: Build

FROM maven:3.9-eclipse-temurin-21 AS build

COPY . .

RUN mvn clean package -DskipTests

#### Stage 2: Runtime

FROM eclipse-temurin:21-jre-jammy

COPY --from=build /target/.jar app.jar

ENTRYPOINT ["java", "-jar", "/app.jar"]

```

10.2. Deploy e Monitoramento

- **Plataforma:** AWS ECS (Elastic Container Service) com Fargate.
- **Deployment:** Preferência por Blue-Green para serviços críticos. O tráfego só é virado após o "Green" passar em todos os Health Checks.
- **Alerting:** Alertas configurados no Prometheus para latência P99 > 500ms e taxa de erro 5xx > 1%.

11. Apêndices e Glossário

11.1. Ferramentas Homologadas

- IDE: IntelliJ IDEA (Recomendado) ou VS Code.
- Gerenciamento de Dependências: Maven 3.8+.
- Banco de Dados: PostgreSQL 15+, Redis 7+.
- Mensageria: RabbitMQ ou AWS SQS.

11.2. Dicionário Técnico (Santo Pegasus)

- Agendio: Nosso sistema core de agendamentos de saúde.
- PegasusPay: Nosso gateway interno de pagamentos multicanal.
- IdP (Identity Provider): Provedor de identidade centralizado.
- Idempotência: Garantia de que uma operação pode ser repetida sem efeitos colaterais indesejados (vital para pagamentos em R\$).

Este documento é de propriedade da Santo Pegasus Soluciones. A divulgação externa sem autorização prévia é proibida.

Arquitetura de Microsserviços e Mapa de Domínios

Santo Pegasus Soluciones

Documento: Arquitetura de Microsserviços e Mapa de Domínios\

Versão: 1.0.0\

Data de Emissão: Junho de 2026\

Departamento: Engenharia de Software / Chapter de Backend\

Classificação: Interno — Uso Técnico Restrito

TABLA DE CONTEÚDOS

- 1. Introdução e Visão Geral da Arquitetura
- 2. Diagrama Textual da Arquitetura Geral
- 3. Catálogo Completo de Microsserviços
- 4. Mapa de Dependências entre Serviços
- 5. Padrões de Comunicação
- 6. Estratégia de Bancos de Dados
- 7. API Gateway
- 8. Infraestrutura na AWS
- 9. Observabilidade Distribuída
- 10. Estratégia de Versionamento de APIs
- 11. Segurança entre Serviços
- 12. Mapa de Squads e Ownership
- 13. Roadmap Técnico
- 14. Architecture Decision Records (ADRs)
- 15. Disposições Finais e Processo de Atualização

SEÇÃO 1 — INTRODUÇÃO E VISÃO GERAL DA ARQUITETURA

1.1 Propósito do Documento

Este documento constitui o mapa arquitetônico oficial da Santo Pegasus Soluciones. Seu propósito é servir como referência canônica para todos os engenheiros, Tech Leads, Product Managers e stakeholders técnicos que precisem compreender como estão organizados, interconectados e implantados os sistemas que compõem o portfólio de produtos da empresa. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

Este artefato deve ser lido em conjunto com o Guia Oficial de Engenharia Backend, que estabelece os padrões de codificação, padrões de design e práticas de segurança obrigatórias para todos os serviços aqui descritos. Ambos os documentos formam o núcleo da base de conhecimento técnico da Santo Pegasus e são fontes de verdade para a tomada de decisões arquitetônicas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

1.2 Contexto Empresarial

A Santo Pegasus Soluciones é uma empresa de tecnologia especializada no desenvolvimento de produtos digitais para o setor de saúde e serviços profissionais. O produto principal da companhia é o Agendio, uma plataforma SaaS de agendamento de consultas médicas que conecta pacientes, médicos e clínicas por meio de uma experiência digital integrada. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

A plataforma Agendio opera em modelo multi-tenant, atendendo a redes de clínicas, hospitais de pequeno e médio porte, e consultórios independentes em todo o Brasil. A criticidade do domínio de saúde exige que a arquitetura priorize disponibilidade, segurança de dados sensíveis, conformidade com a LGPD (Lei Geral de Proteção de Dados) e auditabilidade completa de todas as operações. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

1.3 Por Que Microserviços

A adoção de uma arquitetura de microserviços na Santo Pegasus não foi uma decisão tomada de forma prematura. A empresa começou com um monolito funcional, e a migração para microserviços foi guiada por necessidades reais e imediatas de escala, autonomia de equipes e velocidade de entrega, em consonância com os princípios fundamentais documentados no Guia de Engenharia Backend. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

As motivações concretas que justificaram a transição são:

- **Escalabilidade Independente:** O serviço de agendamento (`agendio-scheduling-service`) apresenta picos de carga nos horários matutinos (entre 07h00 e 09h00) que não impactam outros domínios. Em arquitetura monolítica, escalar esse módulo implicaria escalar toda a aplicação, desperdiçando recursos computacionais e aumentando custos operativos. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]
- **Autonomia de Squads:** Com equipes organizadas por domínio de negócio, a arquitetura de microserviços permite que cada squad tenha ownership completo de seu ciclo de vida de software — desde o código até o deploy em produção — sem coordenação burocrática com outras equipes para releases. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

- **Resiliência e Isolamento de Falhas:** Um defeito crítico no `payment-service` não deve causar a indisponibilidade do agendamento de consultas. O isolamento de processos garante que as falhas sejam contidas dentro do domínio afetado. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

- **Flexibilidade Tecnológica:** Diferentes domínios têm diferentes requisitos técnicos. O `medical-records-service` se beneficia do MongoDB para o armazenamento de documentos clínicos flexíveis e semiestruturados, enquanto o `auth-service` requer a consistência forte do PostgreSQL para a gestão de credenciais. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

- **Velocidade de Entrega:** A independência de deploys permite que múltiplas squads liberem novas funcionalidades no mesmo dia sem coordenação de releases nem janelas de manutenção compartilhadas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

1.4 Princípios Arquitetônicos que Guiam as Decisões

Todas as decisões arquitetônicas na Santo Pegasus estão ancoradas nos seguintes princípios: \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

#	Princípio	Descrição
P-01	High Cohesion, Low Coupling	Cada microserviço deve encapsular um domínio de negócio bem definido. Dependências entre serviços devem ser minimizadas e, quando existentes, preferencialmente assíncronas.
P-02	Database per Service	Cada microserviço possui e controla exclusivamente seu próprio banco de dados. O acesso direto ao banco de dados de outro serviço é terminantemente proibido.
P-03	API First	Contratos de API (OpenAPI/Swagger) são definidos antes da implementação. Isso garante que os consumidores possam desenvolver em paralelo usando mocks.
P-04	Security by Default	Todas as comunicações entre serviços são autenticadas. Nenhum endpoint interno é acessível sem validação de identidade. mTLS é obrigatório para comunicação interna.

P-05	Observability by Design	Logs estruturados, métricas e rastreamento distribuído não são adicionados post-hoc; são parte do scaffolding inicial de cada novo serviço.
P-06	Fail Fast, Recover Gracefully	Os serviços implementam circuit breakers, timeouts e políticas de retry com backoff exponencial para garantir resiliência diante de falhas de dependências.
P-07	Compliance First	Decisões sobre armazenamento, transmissão e processamento de dados sensíveis de saúde são guiadas pela LGPD e boas práticas de segurança antes de qualquer requisito funcional.
P-08	Infrastructure as Code	Toda a infraestrutura AWS é definida como código (Terraform/AWS CDK). Nenhum recurso de produção é criado manualmente via Console.
P-09	Evolutionary Architecture	A arquitetura aceita que mudará. Decisões são tomadas com horizontes de tempo claros e revisadas periodicamente nos fóruns de Architecture Review.
P-10	SOLID in Every Service	Os princípios SOLID de orientação a objetos são aplicados tanto a nível de classe quanto a nível de serviço. Cada microsserviço tem uma única responsabilidade de domínio.

1.5 Ecossistema Tecnológico Principal

O ecossistema tecnológico da Santo Pegasus é padronizado para garantir consistência operacional e reduzir a carga cognitiva das equipes: \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 4\]

- **Linguagem & Runtime: Java 17+ (LTS), com adoção gradual do Java 21 (Virtual Threads).**
- **Framework Principal: Spring Boot 3+, Spring Security, Spring Cloud.**
- **Containerização: Docker (imagens imutáveis), orquestrados no AWS ECS Fargate.**
- **Bancos de Dados: PostgreSQL (relacional), MongoDB/AWS DocumentDB (documentos), Redis/AWS ElastiCache (cache).**
- **Mensageria: AWS SQS (filas e eventos entre domínios).**
- **Autenticação: JWT + OAuth 2.0 / OpenID Connect, gerenciado pelo `auth-service`.**
- **Observabilidade: SLF4J + Logback, Spring Boot Actuator, Micrometer, Prometheus, Datadog.**
- **CI/CD: GitHub Actions + Pipelines com Docker, Flyway/Liquibase para migrations.**

- **Secrets Management: AWS Secrets Manager + Spring Cloud Config.**

SEÇÃO 2 — DIAGRAMA TEXTUAL DA ARQUITETURA GERAL

2.1 Visão de Alto Nível

O diagrama a seguir representa a topologia completa da arquitetura de microsserviços da Santo Pegasus: \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 5\]

...

CLIENTES EXTERNOS

[Web App React] [Mobile App iOS/Android] [Parceiros de API de Clínicas]

▼ HTTPS / TLS 1.3

AWS API GATEWAY (Kong / AWS API GW)

- Routing • Rate Limiting • Auth Token Validation
- SSL Termination • CORS • Request Logging

▼▼▼▼▼

auth- user- agendio- payment- ai-assistant-

service service scheduling service service

:8081 :8082 -service :8085 :8087

:8083

▼▼▼▼▼

[PostgreSQL] [PostgreSQL] [PostgreSQL] [Pinecone DB]

auth_db users_db payments_db (Vector Store)

▼▼▼

agendio- medical- audit-

notif- records- service

service service :8088

:8084 :8086

▼

▼▼ [PostgreSQL]

[AWS SES] [MongoDB / audit_db

[AWS SNS] DocumentDB]

medical_records_db

CAMADA DE MENSAGERIA ASSÍNCRONA (AWS SQS QUEUES)

appointment-created.fifo → [agendio-notification-service]

appointment-cancelled.fifo → [agendio-notification-service] & [audit-service]

payment-confirmed.fifo → [agendio-scheduling-service] & [audit-service]

user-created.fifo → [agendio-notification-service] & [audit-service]
audit-events.fifo → [audit-service]
INFRAESTRUTURA COMPARTILHADA
AWS ElastiCache (Redis) | AWS CloudWatch + Datadog (Observability) | AWS Secrets Manager + Spring Cloud Config
...

2.2 Fluxo de uma Requisição Típica (Agendamento de Consulta)

1. Cliente (App Mobile): `POST /v1/appointments [Bearer JWT]`
2. API Gateway: Valida JWT via `auth-service` (cache Redis), aplica rate limit e roteia para o `agendio-scheduling-service`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]
3. agendio-scheduling-service: Valida disponibilidade do médico, verifica perfil do paciente (REST síncrono), cria registro no `scheduling\_db`, publica evento "appointment-created" no SQS e retorna `201 Created`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]
4. \[Async\] agendio-notification-service: Consome SQS e envia e-mail/SMS de confirmação via AWS SES/SNS. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]
5. \[Async\] audit-service: Consome SQS e registra a ação no `audit\_db` com Trace ID completo. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]

SEÇÃO 3 — CATÁLOGO COMPLETO DE MICROSERVIÇOS

3.1 Resumo do Catálogo \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]

Serviço	Porta	Domínio	Banco de Dados	Squad Owner
auth-service	8081	Identidade & Segurança	PostgreSQL	Squad Hermes
user-service	8082	Usuários & Perfis	PostgreSQL	Squad Hermes
agendio-scheduling-service	8083	Agendamento (Core)	PostgreSQL	Squad Agendio Core
agendio-notificatio n-service	8084	Notificações	— (stateless)	Squad Agendio Core
payment-service	8085	Pagos & Faturamento	PostgreSQL	Squad Pagamentos

medical-records-service	8086	Histórico Clínico	MongoDB	Squad Clínico
ai-assistant-service	8087	Assistência IA (RAG)	Pinecone (Vector)	Squad IA
audit-service	8088	Auditoria & Compliance	PostgreSQL	Squad Governance

3.2 `auth-service` — Autenticação e Autorização

- **Repositório Git:** `git@github.com:santopegasus/auth-service.git`
- **Squad Owner:** Squad Hermes | **Tech Lead:** Isabella Carvalho
- **Responsabilidade:** Raiz de confiança do ecossistema. Emissão, validação e revogação de tokens JWT. Gestão do ciclo de vida de credenciais e fluxos OAuth 2.0 / OIDC. Toda autenticação passa obrigatoriamente por este serviço. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 8\]
- **Tecnologias:** Spring Boot 3.x + Security 6, JWT (RS256), PostgreSQL 15, Redis (Token Cache), Secrets Manager.
- **APIs Expostas:** Login, Refresh, Logout (Blacklist no Redis), OAuth2 Token, Validate (usado pelo Gateway), Password Reset. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 8\]

3.3 `user-service` — Gestão de Perfis de Usuários

- **Responsabilidade:** Gerencia perfis de pacientes, médicos, administradores e operadores. Fonte da verdade para dados de perfil (nome, contato, especialidades). Não armazena credenciais (exclusivo do `auth-service`). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 9\]
- **Tecnologias:** Spring Boot 3.x, PostgreSQL 15, Redis (Cache de Perfil), MapStruct, Swagger UI.
- **Modelo de Dados:** `users`, `doctors`, `patients`, `addresses`, `preferences`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 10\]

3.4 `agendiao-scheduling-service` — Núcleo de Agendamento

- **Responsabilidade:** Serviço mais crítico. Gerencia lógica de criação, confirmação, cancelamento e reagendamento de consultas. Controle de agenda de médicos, conflitos de horários e regras de negócio de clínicas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 10\]
- **Tecnologias:** Spring Boot 3.x + JPA, PostgreSQL 15 (transações ACID), Flyway, Resilience4j.

- Dependências: `user-service` (validar atores), `payment-service` (verificar pagamento), `audit-service` (log de alteração), `agendiao-notification-service` (disparo de avisos). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 11\]

3.5 `agendiao-notification-service` — Notificações

- Responsabilidade: Serviço stateless que orquestra o envio de e-mails (AWS SES) e SMS (AWS SNS) baseados em preferências do usuário consultadas no `user-service`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 12\]
- Tecnologias: Spring Boot 3.x + Spring Cloud AWS, Thymeleaf (Templates HTML), SQS Consumer.

3.6 `payment-service` — Processamento de Pagos

- Responsabilidade: Ciclo de vida de transações financeiras (Pix, Cartão de Crédito/Débito), integração com gateway externo (Stripe), estornos e recibos. Serviço idempotente e completamente auditado. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 13\]
- Tecnologias: Spring Boot 3.x, PostgreSQL 15, Stripe API, Banco Central API (Pix), Assinatura HMAC para Webhooks.

3.7 `medical-records-service` — Histórico Clínico (LGPD Compliance)

- Responsabilidade: Serviço mais sensível (proteção de dados). Gere anamneses, diagnósticos, prescrições e exames. Acesso controlado por RBAC e auditado. Consentimento explícito do paciente é obrigatório. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 14\]
- Tecnologias: Spring Boot 3.x + Security (RBAC granular), MongoDB (AWS DocumentDB), AWS S3 (anexos DICOM/PDF com URLs pré-assinadas), Criptografia em repouso (AWS KMS), Pipeline de anonimização (Art. 18 LGPD). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 14-15\]

3.8 `ai-assistant-service` — Assistente de IA com Arquitetura RAG

- Responsabilidade: Assistente de triagem inteligente para ajudar pacientes a identificar especialidades médicas com base em sintomas. Usa arquitetura RAG (Retrieval-Augmented Generation). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 16\]
- Tecnologias: LangChain4j, OpenAI GPT-4o, Pinecone (Vector Store para conhecimento médico), Redis (Cache de respostas).

3.9 `audit-service` — Registro de Auditoria e Compliance

- **Responsabilidade:** Registro histórico imutável de ações sensíveis. Consumidor puro (recebe dados via SQS). Registros arquivados no S3 Glacier após 90 dias. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 17\]
- **Tecnologias:** PostgreSQL 15 (immutable append-only), AWS S3 Glacier, SQS Consumer exclusivo.

SEÇÃO 4 — MAPA DE DEPENDÊNCIAS ENTRE SERVIÇOS

4.1 Tabela de Dependências \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 18\]

Serviço Consumidor	Serviço Provedor	Tipo	Protocolo	Criticidade
api-gateway	auth-service	Síncrono	REST / Redis Cache	Alta
agendio-scheduling-service	user-service	Síncrono	REST (WebClient)	Alta
agendio-scheduling-service	payment-service	Síncrono	REST (WebClient)	Alta
payment-service	audit-service	Assíncrono	SQS	Média
agendio-scheduling-service	agendio-notification-service	Assíncrono	SQS	Média
user-service	audit-service	Assíncrono	SQS	Média
medical-records-service	audit-service	Assíncrono	SQS	Alta

SEÇÃO 5 — PADRÕES DE COMUNICAÇÃO

5.1 REST Síncrono com WebClient

A Santo Pegasus utiliza Spring WebClient (não-bloqueante/reactivo) para comunicações síncronas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 20\]

- **Regras:** mTLS obrigatório, Timeout de conexão (3s), Timeout de leitura (10s), Circuit Breaker (Resilience4j), Retry com backoff exponencial.

5.2 gRPC para Alta Performance

Adotado para comunicações de alta frequência e baixa latência entre serviços internos (ex: validação rápida de médico no `user-service`). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 20-21\]

5.3 Mensageria Assíncrona com AWS SQS

Padrão preferencial para eventos entre domínios. Utiliza SQS FIFO Queues para garantir ordenação e entrega exatamente-uma-vez (exactly-once delivery). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 21\]

- Padrões: Filas FIFO, DLQ configurada (maxReceiveCount = 3), Idempotência no consumidor, Envelope de evento padronizado.

SEÇÃO 6 — ESTRATÉGIA DE BANCOS DE DADOS

6.1 Database per Service Pattern

Cada microserviço controla exclusivamente sua própria base de dados. Integração entre domínios ocorre apenas via APIs ou mensageria. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 22\]

6.2 Mapa de Bancos de Dados por Serviço \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 22-23\]

- PostgreSQL 15 (RDS Multi-AZ): `auth\_db`, `users\_db`, `scheduling\_db`, `payments\_db`, `audit\_db`.
- MongoDB 6 (DocumentDB): `medical\_records\_db` (esquema flexível).
- Pinecone Cloud: `knowledge\_vectors` (Vector store para RAG).
- Redis 7 (ElastiCache): Cache Global (múltiplos namespaces).

SEÇÃO 7 — API GATEWAY

O API Gateway (Kong) é o único ponto de entrada para clientes externos. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 23\]

- Responsabilidades: Roteamento, Autenticação Centralizada (JWT), Rate Limiting (100 req/min por tenant), SSL/TLS Termination (TLS 1.3), Request Logging com Trace ID, Transformações (Headers contextuais). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 23-24\]

SEÇÃO 8 — INFRAESTRUTURA NA AWS

8.1 Serviços AWS Utilizados \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 25\]

- ECS Fargate: Orquestração de contêineres Docker.
- RDS PostgreSQL Multi-AZ: Bancos relacionais com failover.
- S3 & S3 Glacier: Arquivos de exames e backups de auditoria (retensão 90+ dias).
- Secrets Manager & KMS: Gestão de credenciais e encriptação de dados sensíveis.
- VPC + Private Subnets: Isolamento de rede para microserviços (nunca expostos publicamente).

SEÇÃO 9 — OBSERVABILIDADE DISTRIBUÍDA

Baseada em três pilares: logs, métricas e rastreamento distribuído. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 26\]

- Trace ID: Propagado em headers HTTP ('X-B3-TraceId') e envelopes SQS para rastrear o ciclo de vida completo de uma requisição. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 27\]
- Logging Estruturado: Formato JSON com campos padronizados. Proibido registrar PII (dados pessoais/sensíveis). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 27\]
- Métricas: Spring Boot Actuator + Micrometer. Prometheus coleta via scraping e Datadog recebe via integração nativa. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 27\]

SEÇÃO 10 — ESTRATÉGIA DE VERSIONAMENTO DE APIS

- Versionamento Explícito: Via prefixo no path ('/v1/', '/v2/'). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 28\]
- Processo de Depreciação: 3 fases (Anúncio -> Período de Suporte \[mês 1-6\] -> Sunset \[mês 6+ com HTTP 410 Gone\]). Versões obsoletas permanecem ativas por no mínimo 6 meses. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 29\]

SEÇÃO 11 — SEGURANÇA ENTRE SERVIÇOS

- **mTLS: Comunicação serviço-a-serviço dentro da VPC usa mTLS (Mutual TLS) com certificados rotacionados a cada 90 dias via AWS ACM Private CA. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 30\]**
- **IAM Roles: Cada microserviço opera com um IAM Role exclusivo seguindo o princípio do privilégio mínimo. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 30\]**
- **Scan de Segredos: Ferramentas como `gitleaks` e `trufflehog` bloqueiam commits com segredos expostos. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 31\]**

MANUAL DE ONBOARDING: BEM-VINDO À SANTO PEGASUS SOLUCIONES

1. BEM-VINDO À SANTO PEGASUS: CULTURA E VALORES

Seja muito bem-vindo(a) à Santo Pegasus Soluciones! Estamos entusiasmados por você ter aceitado o desafio de voar conosco. Na Pegasus, não construímos apenas software; criamos soluções escaláveis que impactam milhares de usuários, sempre com o foco na excelência técnica e na colaboração humana.

Nossa História

A Santo Pegasus nasceu em 2018, na garagem de um sobrado no bairro da Vila Madalena, em São Paulo, a partir da inquietação de três fundadores — uma engenheira de dados, um arquiteto de software e uma product designer — que acreditavam que a tecnologia brasileira poderia competir de igual para igual com soluções internacionais. O nome "Pegasus" foi escolhido em homenagem ao cavalo alado da mitologia grega, símbolo de elevação e liberdade; o "Santo" veio de uma brincadeira interna nos primeiros meses da empresa, quando os fundadores diziam que só um "santo milagre" faria aquele MVP funcionar a tempo do primeiro grande cliente. O nome pegou, e a lenda interna se tornou parte da nossa identidade.

Nos primeiros dois anos, a empresa cresceu organicamente atendendo clientes de médio porte no setor de varejo e saúde, sempre com um time pequeno e multidisciplinar. Em 2020, com a aceleração da transformação digital, a Pegasus captou seu primeiro aporte de investimento anjo e iniciou a construção da plataforma "Agendio", nosso produto-carro-chefe de agendamento e gestão de atendimentos, que hoje atende milhares de estabelecimentos em todo o Brasil e começa sua expansão para a América Latina.

Hoje, em 2026, somos mais de 180 "Pegasos" (assim chamamos nossos colaboradores) distribuídos entre o Ninho físico em São Paulo e squads remotas em diversas cidades do Brasil. Continuamos com a mesma inquietação dos fundadores: a convicção de que tecnologia bem-feita, com pessoas bem tratadas, muda o jogo.

Nossa Missão

Capacitar empresas através da tecnologia de ponta, simplificando processos complexos e entregando valor real de forma ágil.

Nossa Visão

Ser reconhecida, até 2030, como a principal plataforma de agendamento, pagamentos e inteligência artificial aplicada para pequenos e médios negócios na América Latina.

Nossos Valores (O "DNA Pegasus")

- **Transparência Radical:** Somos abertos sobre nossos sucessos e, principalmente, sobre nossos erros. O aprendizado vem da honestidade. Isso se reflete em nossos post-mortems sem culpados (blameless), em nossos all-hands mensais onde números reais (bons e ruins) são compartilhados, e na forma como líderes dão e recebem feedback.
- **Qualidade sem Atalhos:** Código bem escrito, testado e documentado não é um luxo, é o nosso padrão. Preferimos entregar um pouco mais tarde e com qualidade do que rápido e quebrado.
- **Colaboração Acima do Ego:** Aqui, a melhor ideia vence, não importa de quem ela venha — seja de um Pegaso Júnior no seu primeiro mês ou de um Staff Engineer com anos de empresa.
- **Equilíbrio (Work-Life Balance):** Valorizamos sua produtividade, mas respeitamos profundamente seu descanso e sua vida pessoal. Reuniões fora do horário comercial são desencorajadas e mensagens fora do expediente não exigem resposta imediata.
- **Ownership (Dono do Problema):** Esperamos que cada Pegaso trate os problemas da empresa como se fossem seus, buscando soluções e não apenas reportando obstáculos.
- **Diversidade como Força:** Acreditamos que times diversos (de gênero, raça, orientação, formação e vivência) constroem produtos melhores e mais inclusivos.

2. ESTRUTURA DE SQUADS E CHAPTERS

Na Santo Pegasus, utilizamos uma estrutura matricial para garantir agilidade na entrega e evolução constante das competências técnicas.

Organograma Textual Simplificado

...

CEO

VP de Engenharia

Head of Engineering (Product)

Squad Agendios Core

Squad Pagamentos

Squad IA/RAG

Squad Plataforma

Head of Chapters (Técnico)

Chapter Back-end

Chapter Front-end

Chapter DevOps/SRE

Chapter Data

Head of People (RH)

Recrutamento & Seleção

Benefícios & Folha

Cultura & Desenvolvimento

...

Squads (Foco no Produto)

Você será alocado em uma Squad multidisciplinar. Cada Squad possui:

- 1 Product Owner (PO)
- 1 Scrum Master / Agile Facilitator
- Desenvolvedores (Backend, Frontend)
- 1 QA (Quality Assurance)
- 1 Product Designer (UX/UI)

Atualmente, temos quatro squads principais de produto:

Squad Agendão Core

Responsável pelo coração da plataforma: o motor de agendamentos, calendários, disponibilidade de horários e gestão de estabelecimentos. É a squad mais antiga da empresa e concentra o maior volume de tráfego. Trabalha intensamente com regras de negócio complexas (fusos horários, recorrência de agendamentos, bloqueios de agenda) e é uma ótima porta de entrada para quem quer entender profundamente o domínio de negócio da Pegasus.

Squad Pagamentos

Cuida de toda a jornada financeira: cobrança recorrente, split de pagamentos, conciliação bancária, emissão de notas fiscais e integração com gateways (Stripe, Pagar.me, PIX). É uma squad com altíssimo rigor de qualidade e segurança, dado o impacto financeiro direto de qualquer falha. Exige atenção especial a testes automatizados e conformidade com PCI-DSS.

Squad IA/RAG

A squad mais nova, criada em 2024, responsável pelo "Pegasus Assistant", nosso assistente virtual baseado em modelos de linguagem (LLMs) com arquitetura RAG (Retrieval-Augmented Generation) para responder dúvidas de clientes finais e auxiliar estabelecimentos na gestão da agenda via linguagem natural. Trabalha com pipelines de embeddings, vetores (pgvector) e orquestração de prompts.

Squad Plataforma

Squad "horizontal" que sustenta as demais: observabilidade, autenticação/autorização, gestão de tenants, billing interno e ferramentas de desenvolvedor (developer experience). Se você gosta de construir "ferramentas para quem constrói ferramentas", essa é a squad ideal.

Chapters (Foco na Carreira e Técnica)

O Chapter é a sua "casa técnica". É onde você troca figurinhas com outros desenvolvedores da mesma disciplina, independentemente da Squad em que estão.

- **Back-end Chapter:** Focado em Java, arquitetura de microsserviços e performance.
- **Front-end Chapter:** Focado em ecossistema JavaScript/TypeScript e design system.
- **DevOps Chapter:** Responsável por Cloud (AWS), CI/CD e infraestrutura como código.
- **Data Chapter:** Focado em engenharia de dados, analytics e LGPD.

Os Chapters se reúnem quinzenalmente para "Refinamentos Técnicos" e mensalmente para "Tech Talks", onde um membro apresenta um tema de aprofundamento técnico ou um estudo de caso de um problema resolvido.

3. DIA 1: SETUP DE ACESSO

Seu primeiro dia será focado em garantir que você tenha as chaves para todas as nossas portas digitais. Abaixo, a tabela completa de acessos, prazos de liberação e responsáveis por cada solicitação.

#	Acesso/Ferramenta	Prazo de Liberação	Solicitar a	Observação
1	E-mail corporativo (Google Workspace)	Antes do Dia 1	TI (automático)	Já vem pronto na sua chegada
2	Slack	Dia 1 (manhã)	TI / Buddy	Convite automático via e-mail corporativo
3	Jira	Dia 1 (manhã)	Scrum Master da Squad	Acesso ao board específico da squad
4	Confluence	Dia 1 (manhã)	Scrum Master da Squad	Base de conhecimento
5	GitHub (org `santo-pegasus-dev`)	Dia 1 (até 12h)	Tech Lead da Squad	Login via SSO com e-mail corporativo
6	VPN (OpenVPN)	Dia 1 (até 17h)	TI (#help-ti)	Arquivo `.ovpn` enviado por e-mail
7	AWS (IAM Identity Center)	Dia 2	DevOps Chapter	Acesso inicial somente leitura
8	Gestor de Senhas (1Password)	Dia 1 (obrigatório)	TI (#help-ti)	Cofre da Squad + cofre pessoal
9	Nexus (repositório interno Maven/NPM)	Dia 2	Tech Lead	Necessário para builds internos

10	Datadog / Grafana (observabilidade)	Dia 3 a 5	Chapter DevOps	Acesso liberado após treinamento básico
11	Figma (apenas Front-end/Design)	Dia 1	Product Designer da Squad	Visualização de protótipos
12	Sistema de Ponto (Ahgora)	Antes do Dia 1	RH	Apenas colaboradores CLT
13	Portal do Colaborador (RH)	Antes do Dia 1	RH	Holerites, férias, benefícios

> **Importante:** Se algum desses acessos não for liberado dentro do prazo, comunique imediatamente seu Buddy ou abra um chamado no canal `#help-ti`.

Ferramentas de Comunicação e Gestão

- Slack:** Nossa sede oficial. Entre nos canais da sua Squad e nos canais gerais (`#announcements`, `#tech-talks`, `#random`).
- Jira:** Onde o trabalho acontece. Você receberá um convite para o board da sua Squad.
- GitHub:** Solicite acesso à organização `santo-pegasus-dev`. O acesso é via SSO com seu e-mail corporativo.
- VPN (OpenVPN):** Essencial para acessar nossos ambientes de staging e bancos de dados internos. O arquivo de configuração será enviado pelo time de TI.

Cloud e Infra

- **AWS:** Você receberá acesso via IAM Identity Center. Verifique se consegue logar no Console AWS (permissões de leitura inicialmente).
- **Confluence:** Nossa base de conhecimento. Comece lendo a documentação técnica da sua Squad específica.

4. CONFIGURAÇÃO DO AMBIENTE BACK-END

Se você é um Pegasus Back-end Developer, siga os passos abaixo. O objetivo é que, ao final do Dia 2, você consiga rodar o projeto principal localmente.

Passo 1: Instalar o SDKMAN e o Java

Utilizamos o SDKMAN para gerenciar versões de Java e ferramentas relacionadas.

```

```bash
curl -s "https://get.sdkman.io" | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
sdk version
...

```

Instale o Amazon Corretto 17, nossa versão padrão:

```
```bash
sdk install java 17.0.9-amzn
sdk default java 17.0.9-amzn
java -version
```
```

## **Passo 2: Instalar Maven**

```
```bash
sdk install maven 3.9.6
mvn -version
```
```

## **Passo 3: Clonar o repositório**

```
```bash
git clone git@github.com:santo-pegasus-dev/agendio-core-service.git
cd agendio-core-service
```
```

> Substitua o nome do repositório pelo projeto principal da sua Squad, informado pelo seu Tech Lead.

## **Passo 4: Instalar o Docker**

Instale o Docker Desktop (ou Colima, no macOS) e verifique:

```
```bash
docker --version
docker compose version
```
```

Suba os serviços de infraestrutura local (PostgreSQL/Redis):

```
```bash
docker compose -f docker-compose.local.yml up -d
docker ps
```
```

## **Passo 5: Configurar as Variáveis de Ambiente**

Nunca versionamos arquivos `.env`. Solicite ao seu Tech Lead o arquivo `.env.example` atualizado, copie-o e preencha os valores:

```
```bash
cp .env.example .env
```
```

Lista de variáveis obrigatórias que você encontrará no `.env`:

| Variável                              | Descrição                                               |
|---------------------------------------|---------------------------------------------------------|
| <code>`DB_HOST`</code>                | Host do banco PostgreSQL local                          |
| <code>`DB_PORT`</code>                | Porta do banco (padrão 5432)                            |
| <code>`DB_NAME`</code>                | Nome do banco de dados                                  |
| <code>`DB_USER`</code>                | Usuário do banco                                        |
| <code>`DB_PASSWORD`</code>            | Senha do banco (fornecida pelo Tech Lead via 1Password) |
| <code>`REDIS_HOST`</code>             | Host do Redis local                                     |
| <code>`REDIS_PORT`</code>             | Porta do Redis (padrão 6379)                            |
| <code>`JWT_SECRET`</code>             | Chave secreta para geração de tokens (ambiente local)   |
| <code>`AWS_REGION`</code>             | Região padrão da AWS (ex: <code>`sa-east-1`</code> )    |
| <code>`AWS_ACCESS_KEY_ID`</code>      | Chave de acesso (perfil de desenvolvimento)             |
| <code>`AWS_SECRET_ACCESS_KEY`</code>  | Chave secreta (perfil de desenvolvimento)               |
| <code>`NEXUS_USERNAME`</code>         | Usuário do repositório Nexus                            |
| <code>`NEXUS_PASSWORD`</code>         | Senha/token do Nexus                                    |
| <code>`SPRING_PROFILES_ACTIVE`</code> | Deve ser <code>`local`</code> para ambiente local       |
| <code>`SENTRY_DSN`</code>             | Endpoint de monitoramento de erros (opcional em local)  |

**Passo 6: Configurar o Maven para o Nexus interno**

Verifique se o seu `~/m2/settings.xml` aponta para o Nexus da Pegasus:

```
``xml
<servers>
 <server>
 <id>pegasus-nexus</id>
 <username>${env.NEXUS_USERNAME}</username>
 <password>${env.NEXUS_PASSWORD}</password>
 </server>
</servers>
``
```

## Passo 7: Rodar a aplicação

```
```bash
mvn clean install -DskipTests
mvn spring-boot:run
```
```

Acesse `http://localhost:8080/actuator/health` e verifique se retorna `{"status":"UP"}`.

## Checklist de ferramentas Back-end

1. **Java 17+:** Utilizamos o Amazon Corretto 17 como padrão.
2. **IDE:** Recomendamos o IntelliJ IDEA (temos licenças Ultimate disponíveis).
3. **Docker:** Instale o Docker Desktop ou Colima. A maioria dos nossos projetos depende de `docker-compose` para subir bancos de dados (PostgreSQL/Redis) localmente.
4. **Variáveis de Ambiente:** Nunca versionamos arquivos `.env`. Solicite ao seu tech lead o arquivo `.env.example` atualizado e configure sua cópia local.
5. **Maven:** Utilizamos o Maven para gestão de dependências. Certifique-se de que o seu `settings.xml` aponta para o nosso Nexus interno se necessário.

---

## 5. CONFIGURAÇÃO DO AMBIENTE FRONT-END

Para os Pegasus Front-end Developers, o setup padrão é:

### Passo 1: Instalar o NVM (Node Version Manager)

```
```bash
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm --version
```
```

### Passo 2: Instalar a versão LTS do Node

```
```bash
nvm install --lts
nvm use --lts
node -v
npm -v
```
```

### Passo 3: Clonar o repositório

```
```bash
git clone git@github.com:santo-pegasus-dev/agendio-web-app.git
```

```
cd agendio-web-app
```

```
...
```

Passo 4: Instalar as dependências

```
```bash
```

```
npm install
```

```
...
```

## **Passo 5: Configurar variáveis de ambiente**

```
```bash
```

```
cp .env.example .env.local
```

```
...
```

Preencha as variáveis principais (`VITE\_API\_BASE\_URL`, `VITE\_AUTH\_DOMAIN`, `VITE\_SENTRY\_DSN`) com os valores fornecidos pelo Tech Lead da sua Squad.

Passo 6: Rodar o projeto em modo de desenvolvimento

```
```bash
```

```
npm run dev
```

```
...
```

Acesse `http://localhost:5173` e verifique se a aplicação carrega corretamente, exibindo a tela de login.

## **Checklist de ferramentas Front-end**

- 1. Node.js:** Utilizamos a versão LTS mais recente. Recomendamos o uso do `nvm` (Node Version Manager).
- 2. IDE:** O VS Code é o padrão de ouro aqui. Recomendamos as extensões: ESLint, Prettier, Tailwind CSS IntelliSense e GitLens.
- 3. Gerenciador de Pacotes:** Utilizamos prioritariamente o `npm`.
- 4. Design System:** Conheça o "Pegasus UI" via Storybook (link disponível no Confluence). Todas as interfaces devem seguir os componentes base.

---

## **6. IDEs: INTELLIJ IDEA E VS CODE**

### **IntelliJ IDEA (Back-end)**

Todos os desenvolvedores Back-end têm direito a licença Ultimate, solicitada via ticket ao TI.  
Plugins obrigatórios:

- **Lombok Plugin:** para suportar as anotações que utilizamos extensivamente.
- **SonarLint:** análise estática de código, alinhada às regras do SonarQube da empresa.
- **CheckStyle-IDEA:** valida o style guide de código Java da Pegasus.

- **.env files support:** realce de sintaxe para arquivos `.env`.
- **Docker Plugin:** gerenciamento de containers direto da IDE.
- **GitToolBox:** informações extras de Git na barra de status.

Configurações recomendadas: ativar "Reformat code" e "Optimize imports" antes de cada commit (Settings > Tools > Actions on Save).

## VS Code (Front-end)

Extensões obrigatórias:

- **ESLint:** validação de regras de lint em tempo real.
- **Prettier – Code formatter:** formatação automática ao salvar.
- **Tailwind CSS IntelliSense:** autocomplete de classes utilitárias.
- **GitLens:** histórico e blame de código inline.
- **Error Lens:** destaque visual de erros e warnings diretamente no editor.
- **EditorConfig for VS Code:** padronização de indentação e encoding.

Configuração recomendada no `settings.json`:

```
``json
{
 "editor.formatOnSave": true,
 "editor.defaultFormatter": "esbenp.prettier-vscode",
 "editor.codeActionsOnSave": {
 "source.fixAll.eslint": true
 }
}

```

## 7. FLUXO DE TRABALHO: GITFLOW E CODE REVIEW

Qualidade é inegociável. Seguimos um fluxo rigoroso para garantir a estabilidade do ambiente de produção.

### GitFlow Pegasus

- **main:** Reflete o código em produção.
- **develop:** Base para novas features.
- **feature/PEG-[ID-DA-TASK]:** Sua branch de trabalho.
- **hotfix/:** Correções críticas em produção.

### Exemplo Prático Completo de uma Feature

Vamos supor que você recebeu a task PEG-1345: "Adicionar campo de observações no agendamento".

### 1. Atualize sua branch develop local:

```
```bash
git checkout develop
git pull origin develop
```
```

### 2. Crie sua branch de feature:

```
```bash
git checkout -b feature/PEG-1345-campo-observacoes-agendamento
```
```

### 3. Desenvolva e faça commits pequenos e descritivos (padrão Conventional Commits):

```
```bash
git add src/main/java/com/pegasus/agendio/model/Agendamento.java
git commit -m "feat(PEG-1345): adiciona campo observacoes na entidade Agendamento"
git add src/main/java/com/pegasus/agendio/dto/AgendamentoDTO.java
git commit -m "feat(PEG-1345): inclui campo observacoes no DTO de resposta"
git add src/test/java/com/pegasus/agendio/AgendamentoServiceTest.java
git commit -m "test(PEG-1345): adiciona testes unitarios para campo observacoes"
```
```

### 4. Suba sua branch para o repositório remoto:

```
```bash
git push origin feature/PEG-1345-campo-observacoes-agendamento
```
```

### 5. Abra o Pull Request no GitHub, sempre apontando para `develop`, preenchendo o template (ver seção abaixo).

### 6. Aguarde as revisões, ajuste conforme comentários, e após 2 aprovações e CI verde, faça o merge (squash merge é o padrão da Pegasus):

```
```bash
```

Merge realizado via interface do GitHub, com opção "Squash and merge"

```
```
```

### 7. Delete a branch local e remota após o merge:

```
```bash
git checkout develop
git pull origin develop
git branch -d feature/PEG-1345-campo-observacoes-agendamento
```
```

## Processo de Pull Request (PR)

1. Abra o PR apontando sempre para `develop`.
2. Preencha o template de PR (descrição, testes realizados, screenshots/vídeos se houver mudança visual).
3. **A Regra de Ouro: Um PR só pode ser mergeado após, no mínimo, 2 aprovações (Approve) de outros desenvolvedores.**
4. O CI (GitHub Actions) deve estar verde (testes passados, lint ok).

## Template Completo de Pull Request

``markdown

### Descrição

Adiciona o campo "observações" à entidade Agendamento, permitindo que o cliente inclua anotações livres no momento do agendamento.

### Task relacionada

PEG-1345

### Tipo de mudança

- ☒ Nova funcionalidade
- ☐ Correção de bug
- ☐ Refatoração
- ☐ Documentação

### Como testar

1. Suba o ambiente local com `docker compose up -d`
2. Rode `mvn spring-boot:run`
3. Faça um POST em `/api/agendamentos` incluindo o campo `observacoes`
4. Verifique se o campo é persistido e retornado corretamente

### Testes realizados

- ☒ Testes unitários adicionados/atualizados
- ☒ Testado localmente
- ☒ Testado em ambiente Staging-Peg

### Screenshots/Vídeos (se aplicável)

N/A (mudança apenas de back-end)

# Checklist

- [x] Meu código segue o style guide da Pegasus
- [x] Realizei uma auto-revisão do meu código
- [x] Comentei partes de lógica complexa
- [x] Atualizei a documentação relevante (Confluence/Swagger)
- [x] Não há warnings novos gerados pelo SonarLint

...

---

## 8. PRIMEIRAS TAREFAS: 30/60/90 DIAS

As expectativas variam conforme o seu nível de senioridade. Use esta seção como guia — converse sempre com seu Tech Lead para ajustar metas específicas da sua Squad.

### Nível Júnior

| Período | Expectativas                                                                                                                       |
|---------|------------------------------------------------------------------------------------------------------------------------------------|
| 30 dias | Ambiente configurado, primeiro commit de batismo mergeado, participação ativa em Dailies, entendimento básico do domínio da Squad. |
| 60 dias | Entrega de 2 a 3 tasks de complexidade baixa/média de forma independente, com pouca necessidade de retrabalho após code review.    |
| 90 dias | Autonomia em tasks de complexidade média, começando a revisar PRs de outros juniores, participação em Refinamentos Técnicos.       |

### Nível Pleno

| Período | Expectativas                                                                                                                  |
|---------|-------------------------------------------------------------------------------------------------------------------------------|
| 30 dias | Ambiente configurado rapidamente, entendimento sólido da arquitetura do produto, primeiras entregas de complexidade média.    |
| 60 dias | Autonomia completa nas tasks da Squad, contribuição em decisões técnicas, revisão de PRs de outros membros.                   |
| 90 dias | Liderança técnica em pequenas iniciativas, participação em discussões de arquitetura, possível início de mentoria a juniores. |

### Nível Sênior

| Período | Expectativas |
|---------|--------------|
|---------|--------------|

|         |                                                                                                                                                |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 30 dias | Diagnóstico crítico da arquitetura atual da Squad, identificação de gaps técnicos e de processo, entrega de pelo menos uma melhoria concreta.  |
| 60 dias | Condução de decisões arquiteturais relevantes, mentoria ativa a plenos e juniores, participação em RFCs (Request for Comments) técnicos.       |
| 90 dias | Responsabilidade por iniciativas estratégicas da Squad, representação do time em discussões entre Chapters, contribuição para roadmap técnico. |

## 9. SUPORTE E PEOPLE (RH): REGRAS E BENEFÍCIOS

A Santo Pegasus opera em conformidade com a legislação brasileira, oferecendo suporte tanto para colaboradores CLT quanto PJ.

### Benefícios Locais

- **Vale-Refeição/Alimentação:** Creditado mensalmente no cartão Caju (Multibenefícios), no valor definido em seu contrato, sempre até o 5º dia útil do mês.
- **Vale-Transporte:** Disponibilizado conforme necessidade (para dias de escritório em SP), solicitado via Portal do Colaborador com pelo menos 10 dias de antecedência.
- **Plano de Saúde:** Bradesco Saúde ou Amil (Top Nacional). Inclusão automática para titulares a partir do primeiro dia; dependentes sob consulta e mediante inclusão formal via RH em até 30 dias da contratação.
- **Plano Odontológico:** Incluso automaticamente junto ao plano de saúde, sem custo adicional para o titular.
- **Gympass:** Ativo para todos os colaboradores após o primeiro dia, com acesso via aplicativo próprio.
- **PLR (Participação nos Lucros e Resultados):** Distribuída anualmente, conforme acordo coletivo e atingimento de metas da empresa e da Squad. O valor de referência e as regras são comunicados em reunião específica de RH no início de cada ciclo, geralmente em janeiro.
- **Auxílio Home Office:** Valor mensal fixo para custos de internet e energia, creditado junto ao vale-alimentação.

### Gestão de Tempo e Férias

- **Ponto (CLT):** Registro via aplicativo Ahgora.
- **Férias:** Devem ser solicitadas via portal do RH com no mínimo 30 dias de antecedência, após alinhamento com o seu gestor de Squad.
- **Reembolsos:** Despesas com cursos, certificações ou hardware devem ser aprovadas previamente e enviadas via relatório de despesas até o dia 20 de cada mês.

## Política de Férias (CLT)

Conforme a legislação brasileira, todo colaborador CLT tem direito a 30 dias corridos de férias após completar 12 meses de trabalho (período aquisitivo). Na Santo Pegasus:

- As férias podem ser fracionadas em até 3 períodos, sendo que um deles não pode ser inferior a 14 dias corridos, e os demais não podem ser inferiores a 5 dias corridos cada.
- É obrigatório o gozo de pelo menos um período de férias dentro dos 12 meses subsequentes ao período aquisitivo, evitando o "vencimento" das férias (o que gera pagamento em dobro, conforme a CLT).
- Solicitações de fracionamento devem ser aprovadas pelo gestor da Squad e formalizadas no Portal do Colaborador.
- O pagamento do adicional de 1/3 constitucional é feito junto com o salário do mês anterior ao início das férias.

## Política de Home Office e Trabalho Híbrido

A Santo Pegasus adota o modelo híbrido flexível como padrão:

- Colaboradores de squads de produto (Agendão Core, Pagamentos, IA/RAG, Plataforma) têm liberdade para escolher seus dias de comparecimento ao Ninho, com recomendação de pelo menos 1 dia por semana presencial para fortalecer a colaboração e integração entre squads.
- Eventos específicos (Sprint Planning trimestral, Tech Talks, integrações de novos Pegasos) podem exigir presença obrigatória, sempre comunicada com antecedência mínima de 7 dias.
- Colaboradores 100% remotos (fora da Grande São Paulo) seguem o mesmo padrão de entregas e participam normalmente de todos os rituais via videoconferência.
- É de responsabilidade do colaborador garantir conexão de internet estável e um ambiente de trabalho adequado (ergonomia, silêncio para reuniões).

---

## 10. SEGURANÇA DESDE O PRIMEIRO DIA

Segurança da informação é responsabilidade de todos os Pegasos, não apenas do time de TI/Segurança.

### Gestor de Senhas Corporativo

- Todo colaborador deve configurar o 1Password já no Dia 1. Nenhuma senha de sistemas internos, banco de dados ou serviços de terceiros deve ser armazenada em blocos de notas, planilhas ou aplicativos pessoais.
- Cada Squad possui um "cofre" (vault) compartilhado no 1Password com credenciais de uso comum (ex: acessos a ambientes de staging).

- Credenciais individuais (ex: chaves de API pessoais) devem ficar no cofre privado do colaborador.

## Política de Dispositivos

- Notebooks corporativos são fornecidos com criptografia de disco ativada (BitLocker/FileVault) e antivírus corporativo instalado por padrão.
- É proibido o uso de dispositivos pessoais para acessar sistemas de produção ou dados sensíveis de clientes.
- **Toda perda ou roubo de equipamento deve ser reportada imediatamente ao canal `#help-ti`, para bloqueio remoto dos acessos.**
- Autenticação multifator (MFA) é obrigatória em todos os sistemas críticos (AWS, GitHub, Google Workspace, 1Password).
- Colaboradores devem manter o sistema operacional e aplicativos sempre atualizados, aplicando patches de segurança assim que notificados pelo TI.

## Boas Práticas de Segurança de Código

- Nunca versionar segredos, chaves ou senhas em repositórios Git (mesmo privados).
- Utilizar sempre o `.gitignore` padrão da Pegasus, que já bloqueia arquivos sensíveis comuns.
- Reportar qualquer vulnerabilidade encontrada via canal `#security-reports`, nunca divulgando publicamente antes da correção.

---

## 11. CHECKLIST DA PRIMEIRA SEMANA

- ☐ Dia 1: Acessar Slack, E-mail, Jira e GitHub. (Validado por: Buddy)
- ☐ Dia 1: Configurar o gestor de senhas 1Password. (Validado por: TI)
- ☐ Dia 1: Participar da reunião de boas-vindas com o RH. (Validado por: RH)
- ☐ Dia 1: Receber e conferir o notebook corporativo e demais equipamentos. (Validado por: TI)
- ☐ Dia 2: Realizar o setup da máquina e rodar o projeto localmente. (Validado por: Tech Lead)
- ☐ Dia 2: Reunião 1:1 com o seu Tech Lead para entender as metas da Squad. (Validado por: Tech Lead)
- ☐ Dia 2: Obter acesso à AWS (IAM Identity Center) em modo leitura. (Validado por: Chapter DevOps)
- ☐ Dia 3: Ler a documentação de arquitetura no Confluence. (Validado por: Buddy)
- ☐ Dia 3: Configurar IDE (IntelliJ ou VS Code) com todos os plugins/extensões obrigatórios. (Validado por: você mesmo, checklist da Seção 6)

- [ ] Dia 4: Realizar o primeiro "Commit de Batismo" (geralmente uma task de baixa complexidade ou correção de documentação). (Validado por: Tech Lead via aprovação do PR)
- [ ] Dia 5: Participar da primeira Daily e Retro da Squad. (Validado por: Scrum Master)
- [ ] Dia 5: Reunião de fechamento de semana com o Buddy para tirar dúvidas gerais. (Validado por: Buddy)

---

## 12. CONTATOS ÚTEIS

| Área/Squad               | Nome/Função                        | Canal Slack           |
|--------------------------|------------------------------------|-----------------------|
| Tech Lead — Agendio Core | Responsável técnico da squad       | `#squad-agendio-core` |
| Tech Lead — Pagamentos   | Responsável técnico da squad       | `#squad-pagamentos`   |
| Tech Lead — IA/RAG       | Responsável técnico da squad       | `#squad-ia-rag`       |
| Tech Lead — Plataforma   | Responsável técnico da squad       | `#squad-plataforma`   |
| Chapter Back-end         | Líder do chapter                   | `#chapter-backend`    |
| Chapter Front-end        | Líder do chapter                   | `#chapter-frontend`   |
| Chapter DevOps/SRE       | Líder do chapter                   | `#chapter-devops`     |
| Chapter Data             | Líder do chapter                   | `#chapter-data`       |
| RH / People              | Time de People                     | `#help-rh`            |
| TI / Suporte Técnico     | Time de Infraestrutura             | `#help-ti`            |
| Segurança da Informação  | Time de Segurança                  | `#security-reports`   |
| Seu Buddy                | Definido no convite de boas-vindas | Mensagem direta (DM)  |

---

## 13. DICIONÁRIO DE TERMOS INTERNOS DA PEGASUS

Para você não ficar perdido nas reuniões, aqui estão os termos mais comuns:

- **"Pegaso":** Como chamamos carinhosamente os colaboradores da empresa.
- **"Voo":** Lançamento de uma grande feature ou nova versão do produto. ("O voo do módulo fiscal será amanhã").
- **"Ninho":** Nosso escritório físico em São Paulo.
- **"Staging-Peg":** Nosso ambiente de homologação que espelha produção.

- **"Refinamento Técnico"**: Reunião onde o Chapter avalia a viabilidade de uma solução antes de ir para o board da Squad.
- **"Debate de Código"**: Uma sessão de pair programming ou discussão técnica intensa.
- **"Commit de Batismo"**: O primeiro commit mergeado por um novo Pegaso.
- **"Buddy"**: Colaborador designado para acompanhar o novo Pegaso nos primeiros 15 dias.
- **"All Hands"**: Reunião geral mensal com todos os Pegasos, onde resultados e novidades são compartilhados.
- **"PEG-[número]"**: Prefixo padrão de todas as tasks no Jira.
- **"RFC" (Request for Comments)**: Documento técnico proposto por alguém do time para discussão aberta antes de decisões arquiteturais importantes.
- **"War Room"**: Canal/sala de guerra criado durante incidentes críticos em produção.
- **"Pegasus Assistant"**: Nosso assistente virtual de IA para clientes finais, desenvolvido pela Squad IA/RAG.
- **"Agendio"**: Nome do nosso produto principal de agendamento e gestão de atendimentos.
- **"DoR" (Definition of Ready)**: Critérios que uma task precisa cumprir antes de entrar no Sprint.
- **"DoD" (Definition of Done)**: Critérios que uma task precisa cumprir para ser considerada finalizada.
- **"Squash Merge"**: Estratégia padrão de merge da Pegasus, que condensa todos os commits de uma feature em um único commit na branch de destino.
- **"Blameless Post-Mortem"**: Análise de incidentes sem apontar culpados, focada em processos e aprendizado.
- **"Tech Talk"**: Apresentação técnica mensal aberta a todos os Chapters.
- **"Sprint Zero"**: Primeira semana de um novo Pegaso, focada em onboarding e ambientação, sem entregas de negócio esperadas.
- **"1:1" (One-on-one)**: Reunião individual periódica entre colaborador e gestor.
- **"Cofre" (1Password)**: Conjunto de credenciais compartilhadas por Squad ou de uso pessoal.
- **"Sede Digital"**: Referência carinhosa ao Slack como ambiente central de comunicação.

---

## 14. PERGUNTAS FREQUENTES DO ONBOARDING

**Não recebi um dos acessos dentro do prazo. O que faço?**

Abra um chamado no canal `#help-ti` informando qual acesso está pendente e há quanto tempo. Se não houver resposta em 24h, avise seu Buddy ou Tech Lead.

**Posso escolher entre IntelliJ e VS Code mesmo sendo Back-end?**

O padrão recomendado para Back-end é o IntelliJ IDEA por conta da integração nativa com Java/Maven/Spring, mas não há bloqueio técnico ao uso de outra IDE, desde que você garanta a aderência aos plugins de qualidade (SonarLint, CheckStyle).

### **Como funciona o pagamento do vale-alimentação no mês de contratação?**

O valor é calculado proporcionalmente aos dias trabalhados no mês de admissão e creditado junto com o próximo ciclo normal do cartão Caju.

### **Posso emendar as férias fracionadas com feriados?**

Sim, desde que respeitados os períodos mínimos legais (um período de ao menos 14 dias e os demais de ao menos 5 dias) e aprovação do gestor da Squad.

### **Sou PJ. Tenho direito aos mesmos benefícios de um CLT?**

Colaboradores PJ têm acesso a benefícios equivalentes conforme definido em contrato específico (ex: vale-alimentação e plano de saúde geralmente inclusos), mas regras de férias e ponto seguem a legislação de prestação de serviços, não a CLT. Consulte seu contrato ou o RH para detalhes.

### **Quantas aprovações preciso para mergear um PR?**

No mínimo 2 aprovações de outros desenvolvedores, além do CI verde (testes e lint aprovados).

### **Meu computador apresentou um problema de segurança (perda, roubo, malware). O que faço?**

Comunique imediatamente o canal `#help-ti` para bloqueio remoto dos acessos corporativos, independentemente do horário.

### **Existe alguma flexibilidade na exigência de 1 dia presencial por semana?**

Sim. A recomendação é flexível e pode ser ajustada com o gestor da Squad conforme necessidades específicas do time e do momento de projeto.

---

**Dúvidas? Procure seu Buddy (o desenvolvedor designado para te acompanhar nos primeiros 15 dias) ou chame o time de People no canal `#help-rh`.**

**Vamos voar alto juntos!**

# **Protocolo Integral e Extenso de Resposta a Incidentes e Engenharia de Confiabilidade (SRE) — Santo Pegasus Soluciones**

## **1. Filosofia SRE e a Gênese da Confiabilidade sob a Ótica da Santo Pegasus Soluciones**

Na Santo Pegasus Soluciones, a confiabilidade não é meramente uma característica desejável do software ou um requisito não-funcional secundário; ela é uma pré-condição absoluta para a existência, continuidade e viabilidade ética do negócio. Operando no ecossistema de HealthTech brasileiro, os sistemas que desenvolvemos e mantemos são o tecido conectivo entre pacientes, médicos, clínicas, laboratórios e instâncias regulatórias. Uma falha no nosso sistema não significa apenas que um usuário não conseguiu ver uma foto ou enviar uma mensagem; significa que um cirurgião não conseguiu acessar um prontuário crítico antes de uma operação, ou que uma ambulância não recebeu as coordenadas corretas através dos nossos sistemas de despacho.

No cenário tecnológico do Brasil, a engenharia de software e a confiabilidade de sistemas (Site Reliability Engineering - SRE) enfrentam desafios singulares que exigem uma adaptação profunda e pragmática dos conceitos originalmente criados pelo Google em contextos de infraestrutura global hipere-escalada. A nossa realidade nacional é caracterizada por flutuações constantes e severas na latência de rede inter-regional. O Brasil possui dimensões continentais, e a infraestrutura de fibra óptica, embora em expansão, apresenta gargalos conhecidos. Por exemplo, rotas que cruzam do Sudeste para o Nordeste muitas vezes sofrem com instabilidades intermitentes em backbones de telecomunicações, dependendo fortemente do IX.br (Brasil Internet Exchange). Uma indisponibilidade parcial no PTT-SP (Ponto de Troca de Tráfego de São Paulo) afeta diretamente a conectividade de milhares de clínicas e hospitais em outros estados que dependem das nossas zonas de disponibilidade específicas na AWS (sa-east-1).

*A Santo Pegasus adota a mentalidade fundamental de que sistemas complexos são inerentemente propensos a falhas. A entropia de software é uma realidade inegável. Portanto, a nossa engenharia de confiabilidade não busca a utopia inalcançável do "zero erro" — uma meta que, historicamente, apenas inibe a inovação e o lançamento de novas funcionalidades —, mas sim o design arquitetônico para degradação elegante e recuperação rápida. O objetivo do SRE na Santo Pegasus é garantir que o usuário final, seja ele um médico realizando uma prescrição controlada de medicamentos ou um paciente agendando uma consulta oncológica crítica, não perceba as falhas subjacentes na infraestrutura. Se um cluster de banco de dados sofrer um failover no meio da noite, ou se um microsserviço de notificação reiniciar devido a um pico de memória, o sistema deve mascarar essas falhas através de retries idempotentes, circuit breakers e fallbacks assíncronos.*

*O SRE, portanto, atua como a ponte estratégica entre a agilidade no desenvolvimento de novos produtos (o imperativo do time de engenharia de software) e a necessidade inegociável de estabilidade operacional (o imperativo do negócio e da regulamentação). O papel do SRE é crucial para assegurar que a velocidade de lançamento de novas features no módulo "Agendio" não comprometa, sob hipótese alguma, a integridade e o sigilo dos dados. Isto é especialmente crítico no contexto da Lei Geral de Proteção de Dados (LGPD - Lei nº*

*13.709/2018), que classifica os dados de saúde como dados pessoais sensíveis, sujeitando as empresas a multas devastadoras e à responsabilização civil no sensível ecossistema de saúde brasileiro. A nossa filosofia é clara: a confiabilidade é a nossa funcionalidade mais importante. Sem ela, o código mais elegante não tem valor.*

## **2. Cultura Blameless (Sem Culpa) e a Psicologia do Erro Sistêmico**

*A base inabalável da resiliência e do aprendizado contínuo na Santo Pegasus é fundamentada na filosofia Blameless (Sem Culpa). Esta não é apenas uma declaração de intenções do setor de Recursos Humanos, mas sim um framework técnico, metodológico e operacional projetado para maximizar a extração de inteligência a partir de falhas no sistema. A organização compreende cientificamente que o erro humano nunca é a causa raiz de um incidente; ele é apenas o catalisador ou um sintoma de deficiências muito mais profundas no design do sistema, nos processos de deployment, ou nas ferramentas fornecidas aos indivíduos.*

*Punir indivíduos por falhas ocorridas em sistemas altamente complexos e acoplados é uma estratégia corporativa falida e contraproducente. Quando engenheiros são repreendidos por cometerem erros, a organização acidentalmente incentiva a cultura do silêncio, da ocultação de falhas e da engenharia defensiva. Se um desenvolvedor teme perder o emprego porque derrubou o banco de dados de produção, ele hesitará em reportar a anomalia, tentará consertá-la secretamente ou atrasará o escalonamento do problema. Esse atraso é fatal. Ele aumenta drasticamente o Tempo Médio de Reparo (MTTR - Mean Time To Repair) e degrada a confiança mútua que é absolutamente essencial para operações de alta performance em momentos de estresse extremo.*

*Na Santo Pegasus, o foco das investigações de incidentes (conduzidas através do documento de Post-Mortem) muda invariavelmente da pergunta acusatória "Quem cometeu o erro?" para a investigação sistêmica: "Por que o sistema permitiu que essa ação específica causasse tal impacto?"*

*Para fins educativos do nosso agente de IA, consideremos o seguinte cenário prático: Se, sob a pressão de um deploy durante a madrugada, um engenheiro júnior executa um comando que deleta acidentalmente uma tabela crítica de agendamentos no banco de dados de produção (PostgreSQL). Em culturas tradicionais, o engenheiro seria demitido. Na cultura Blameless da Santo Pegasus, a investigação determinará que a falha real e inaceitável reside nos seguintes pontos sistêmicos:*

**1. Ausência de Segregação de Privilégios (IAM):** Por que as credenciais fornecidas ao engenheiro tinham permissão de `DROP TABLE` no ambiente de produção?

**2. Falta de Guardrails Automatizados:** Por que o pipeline de CI/CD não impediu a execução direta de comandos DDL na infraestrutura, exigindo que todas as mudanças passassem por ferramentas de migração versionadas como Liquibase ou Flyway?

**3. Ergonomia das Ferramentas:** A ferramenta de CLI utilizada induzia ao erro? Estava claro que o contexto ativo era `production` e não `staging`?

Cultivar um ambiente de segurança psicológica — onde qualquer colaborador, independentemente do cargo, nível de antiguidade (júnior, pleno, sênior) ou tempo de empresa, sinta-se ativamente encorajado a declarar um incidente (acionar o alarme) imediatamente ao percebê-lo, sem qualquer medo de retaliação — é um pilar estratégico não-negociável da Santo Pegasus.

Ocultar um erro deliberadamente ou tentar resolvê-lo secretamente para evitar escrutínio da liderança é, paradoxalmente, considerado o único erro real e passível de demissão dentro da organização. Acreditamos que a luz solar é o melhor desinfetante tecnológico. Quando os erros são trazidos à tona rapidamente, a equipe de SRE pode intervir, mitigar o impacto, aprender com a vulnerabilidade descoberta e implementar proteções em nível de arquitetura (como infraestrutura imutável e testes de caos) para garantir que aquele erro humano específico seja impossível de ser cometido novamente no futuro.

### **3. Taxonomia Exaustiva de Incidentes no Ecossistema Agendio (Foco em HealthTech)**

A precisão terminológica é o alicerce de uma resposta eficiente. Durante uma crise técnica, a ambiguidade na comunicação custa minutos preciosos que podem resultar em perdas financeiras ou riscos à vida humana. Na Santo Pegasus, diferenciamos rigorosamente os eventos operacionais para minimizar o ruído nos sistemas de monitoramento (alert fatigue) e garantir que a capacidade cognitiva e o esforço de engenharia sejam aplicados exclusivamente onde há risco real ao ecossistema Agendio.

Um Incidente é definido tecnicamente como qualquer interrupção não planejada ou redução mensurável na qualidade de um serviço de TI. Diferente de um e-commerce onde uma lentidão resulta em abandono de carrinho, no contexto de HealthTech, a latência é um bloqueio de atendimento. Um incidente não precisa ser uma queda total (Downtime). Por exemplo, um aumento súbito de erros HTTP 500 no módulo de agendamento, ou uma degradação no 99º percentil (p99) de latência, configuram incidentes ativos.

Um Problema refere-se à causa subjacente, oculta e persistente de um ou mais incidentes, como um memory leak (vazamento de memória) crônico em um microsserviço de prontuário eletrônico rodando em Node.js, ou conexões órfãs (zombie connections) não encerradas adequadamente no PostgreSQL (PgBouncer). Eventos são ocorrências normais que indicam o funcionamento do sistema (ex: picos previsíveis de acesso às segundas-feiras de manhã), enquanto Mudanças Planejadas são alterações autorizadas, documentadas e versionadas, como janelas de manutenção na calada da noite.

No contexto crítico da saúde brasileira, a nossa taxonomia de incidentes mapeia cenários com repercussões físicas. Exemplos práticos para o treinamento do Agente de IA:

- **Falha em APIs da ANS (Padrão TISS/TUSS):** A interrupção da API de integração com operadoras de saúde inviabiliza verificações de elegibilidade e autorização de exames em tempo real. Se o sistema não consegue validar a carteira do plano de saúde, o paciente na recepção do hospital não é atendido.
- **Degradação no Sistema de Prescrição:** Uma latência superior a 3 segundos na busca da base de dados de medicamentos (via elasticsearch) é tratada como um incidente grave. Médicos em pronto-socorros (UPAs) realizam dezenas de prescrições por hora; sistemas lentos causam superlotação nas salas de espera.
- **Interrupção PACS/DICOM:** A indisponibilidade dos sistemas de comunicação e arquivamento de imagens médicas (PACS) impede que neurologistas e cirurgiões visualizem tomografias essenciais para intervenções de emergência (ex: pacientes com AVC).
- **Desconexão IoT em Telemetria de UTI:** Falhas de rede que afetam o protocolo MQTT utilizado para transmitir sinais vitais de monitores de UTI para os dashboards centrais de enfermagem.
- **Erros em Assinatura Digital (ICP-Brasil):** Falhas na geração de QR Codes e validação de tokens para receitas de medicamentos controlados, o que bloqueia legalmente a dispensação do medicamento na farmácia.

## **4. Matriz de Severidade Brasileira (SEV) e Impactos Multidimensionais (Financeiro e Legal)**

A severidade de um incidente (SEV) na Santo Pegasus não é definida apenas pelo volume de requisições perdidas, mas através de uma matriz multidimensional que integra o impacto no usuário final, perdas financeiras diretas para as clínicas parceiras, risco clínico e, criticamente, os riscos de conformidade jurídica, com foco especial nas exigências da Lei Geral de Proteção de Dados (LGPD) e na fiscalização da Autoridade Nacional de Proteção de Dados (ANPD).

## **SEV-1 (Crítico - Intervenção Imediata e Total)**

- **Definição Técnica:** O serviço core ou uma funcionalidade vital está totalmente indisponível para mais de 10% da base de usuários, ou há um vazamento ativo de dados sensíveis. Nenhuma solução de contorno (workaround) está disponível.
- **Impacto Financeiro:** Projetado acima de R\$ 100.000 por hora de inatividade devido a cirurgias canceladas, glosas médicas e impossibilidade de faturamento.
- **Risco Clínico e Jurídico:** Risco iminente de fatalidade ou agravamento do quadro de saúde de pacientes. No caso de vazamento de prontuários (dados sensíveis, Artigo 11 da LGPD), o risco jurídico é máximo. Isso pode resultar em multas aplicadas pela ANPD de até 2% do faturamento líquido da empresa, limitadas a absurdos R\$ 50 milhões por infração (Art. 52, LGPD), além de ações coletivas de danos morais movidas pelo Ministério Público ou Idec. O SLA de notificação à ANPD é de 48 horas úteis, tornando a resposta SEV-1 uma operação com escrutínio legal.

## **SEV-2 (Alto - Prioridade Máxima de Engenharia)**

- **Definição Técnica:** Degradação severa de funcionalidades críticas ou indisponibilidade parcial contida em certas regiões (ex: falha apenas em servidores que atendem a região Sul). O sistema é utilizável, mas a experiência é inaceitável.
- **Impacto Financeiro:** Perdas estimadas variam entre R\$ 20.000 e R\$ 100.000 por hora.
- **Risco Clínico e Jurídico:** Risco moderado de judicialização. O atendimento médico é severamente atrasado, mas não completamente bloqueado (ex: médicos voltam a usar receituário de papel temporariamente).

## **SEV-3 (Médio - Mitigação em Horário Comercial)**

- **Definição Técnica:** Falhas em componentes secundários, relatórios analíticos, ou bugs de interface (UI) que afetam grupos isolados. As transações principais (agendamento, prescrição) ocorrem normalmente.
- **Impacto:** Prejuízo financeiro inferior a R\$ 20.000 por hora. Sem risco clínico ou de vazamento de dados. Exige monitoramento rigoroso para garantir que não exista "efeito cascata" que o eleve a SEV-2.

## SEV-4 (Baixo - Tratado no Backlog)

- **Definição Técnica:** Pequenos defeitos, erros de digitação em documentação, ou falhas em serviços internos (ex: painel de RH da própria Santo Pegasus). Não afeta a jornada do paciente ou do médico e é resolvido no próximo Sprint do time de engenharia.

## 5. Governança Avançada de Crise e Gestão de Fadiga (Protocolo ICS)

Para gerenciar o caos técnico de incidentes SEV-1 e SEV-2, a Santo Pegasus adaptou o Sistema de Comando de Incidentes (ICS - Incident Command System), estruturando hierarquias temporárias focadas em eficiência militar e redução de carga cognitiva.

Em uma "War Room" (Sala de Guerra), a estrutura tradicional da empresa (CEOs, Diretores, Gerentes) é abolida. A hierarquia durante o incidente passa a ser baseada em funções operacionais estritas:

**1. Incident Commander (IC):** É a autoridade suprema e incontestável do incidente. O IC não toca em código, não executa queries no banco e não acessa painéis da AWS. Sua única função é delegar tarefas, gerenciar o estado global da crise, tomar decisões executivas sobre trade-offs (ex: "Desligue o módulo de pagamentos para salvar o banco de dados principal de um crash total") e garantir que a comunicação flua. Se o CEO da empresa entrar na War Room, ele se subordina às ordens do IC.

**2. Tech Lead / Subject Matter Expert (TL/SME):** É o braço tático. São os engenheiros especialistas investigando os logs no Datadog, analisando traces, reiniciando pods no Kubernetes ou elaborando planos de rollback no Terraform. Eles respondem apenas ao IC.

**3. Communications Lead (CL):** É o escudo protetor da equipe técnica. O CL lida com todas as perguntas de partes interessadas externas (Stakeholders, clientes, diretores) e atualiza a Status Page. Ele protege os engenheiros da clássica e nociva interrupção gerencial: "Já voltou? Quanto tempo falta?".

**4. Scribe (Escriba):** Mantém um log minuto a minuto de todas as ações tomadas, hipóteses testadas (mesmo as que falharam) e comandos executados. Este registro é o esqueleto do Post-Mortem subsequente.

**Etiqueta de Comunicação em Malha Fechada (Closed-loop Communication):** Para evitar ruídos, as comunicações na sala virtual seguem protocolos de confirmação mútua para evitar ambiguidades. Erro: "Vou reiniciar o servidor." (Qual servidor? Quem vai reiniciar? Quando?) Correto: "IC, aqui é o DBA. Solicitando permissão para reiniciar as instâncias Read-Replica do cluster principal de agendamento." -> IC Responde: "DBA, permissão concedida para reiniciar as Read-Replikas. Informe quando concluído."

**Gestão de Fadiga (Burnout Prevention):** Um aspecto ignorado por startups, mas central na Santo Pegasus, é a falibilidade humana sob exaustão. O Incident Commander tem a autoridade médica e o dever operacional de ordenar pausas obrigatórias para descanso mental em crises prolongadas. Se um incidente ultrapassar a marca de 8 (oito) horas consecutivas, o IC deve organizar obrigatoriamente a substituição completa de toda a equipe técnica na War Room (handover estruturado). Engenheiros exaustos tomam decisões prejudiciais, pulam etapas de segurança e inserem novos bugs na tentativa de consertar os antigos.

## **6. Política de On-Call, Conformidade com a CLT e Saúde Mental Sustentável**

Sistemas de saúde operam 24x7x365. Para suportar essa realidade sem destruir a saúde mental dos nossos engenheiros, o regime de plantão (On-Call) da Santo Pegasus é estruturado sob duas premissas absolutas: o rigor matemático na divisão de carga e a conformidade hermética com a Consolidação das Leis do Trabalho (CLT) do Brasil.

**Regulamentação e Remuneração (Artigo 244 da CLT):** A cultura de "trabalho por amor" em startups frequentemente esconde abusos trabalhistas disfarçados de plantões informais. Na Santo Pegasus, todo engenheiro escalado no PagerDuty fora de seu horário de expediente encontra-se em regime legal de Sobreaviso. Conforme o Art. 244, §2º da CLT, o trabalhador que permanece em sua residência aguardando um eventual chamado (via PagerDuty, Slack ou SMS) é remunerado à razão de 1/3 (um terço) do valor da sua hora normal de trabalho por cada hora de prontidão, mesmo que não haja nenhum incidente no período.

No instante em que o alarme dispara e o engenheiro efetua o "Acknowledge" (reconhecimento do incidente) no sistema, iniciando o trabalho efetivo de mitigação no seu laptop, o regime transaciona instantaneamente. A remuneração passa a ser contada como hora extra integral (100%), acrescida de todos os adicionais previstos em convenções coletivas (Sindpd). Se a atuação ocorrer entre as 22h00 e as 05h00, incide automaticamente o Adicional Noturno (acréscimo de 20%, e a hora computada como 52 minutos e 30 segundos).

**Diretrizes de Proteção contra Burnout e Redução de Toil:**

- **Proibição do "Hero Culture":** Escalas de um único indivíduo sendo responsável pelo sistema a semana inteira são proibidas. A rotação mínima on-call deve ter 3 a 4 engenheiros habilitados para garantir ciclos saudáveis de descanso ininterrupto (ex: 1 semana on-call seguida de 3 semanas off-call).
- **Descanso Intersticial:** Se um engenheiro é acionado às 03h00 da manhã para resolver um SEV-1 que se estende até as 06h00, ele está legalmente e culturalmente isento de comparecer ao Daily Scrum às 09h00. A política dita que ele tenha um período mínimo de 11 horas de repouso consecutivo (Art. 66 da CLT) antes de retomar atividades rotineiras.

- **Toil Management (Trabalho Massante):** O SRE dita que o acionamento humano fora de hora só é válido se a intervenção exigir intelecto humano genuíno. Se o alerta diz "O disco C: atingiu 95%, limpe os logs", isso é toil. Alertas acionáveis requerem automação (ex: um script AWS Lambda que esvazia a lixeira automaticamente). O objetivo anual de engenharia é que 50% do tempo do time SRE seja gasto em melhorias arquiteturais, e não apagando incêndios previsíveis.

## 7. Fluxo de Resposta a Incidentes: Mitigação vs. Resolução (OODA Loop)

A resposta a incidentes na Santo Pegasus não é um esforço caótico de tentativa e erro, mas um processo militarizado baseado no ciclo OODA (Observe, Orient, Decide, Act). O aspecto mais crucial deste fluxo é a separação hermética entre duas fases: a Mitigação (estabilização imediata para o paciente/médico) e a Resolução (correção definitiva do bug no código).

**1. Detecção (Observe):** O incidente é capturado pelo monitoramento sintético ou por um alarme de p99 de latência no Prometheus. O pager dispara.

**2. Triage e Declaração (Orient):** O engenheiro on-call avalia o impacto (Blast Radius) em menos de 5 minutos. Se confirmar que o módulo de prescrição está fora, ele declara um SEV-1, aciona o Incident Commander e abre a War Room.

**3. Mitigação (Decide & Act):** Esta é a etapa mais crítica. Na Santo Pegasus, a mitigação prioriza a continuidade do serviço acima de qualquer investigação forense. Se um deploy recente causou instabilidade, a ação imediata não é debugar os logs para entender o porquê da falha; a ação imediata é o rollback. O objetivo da mitigação é devolver uma funcionalidade aceitável ao usuário final o mais rápido possível, interrompendo o sangramento financeiro e o risco clínico.

**4. Resolução:** Apenas quando o sistema volta ao estado de operação normal e os usuários estão sendo atendidos (ainda que rodando uma versão anterior do sistema), os engenheiros iniciam a investigação profunda, acessando logs de replicação, heap dumps e traces no Datadog.

É importante notar que um incidente pode estar mitigado, mas ainda não resolvido. Exemplo: um ataque DDoS inunda nosso servidor de login. A mitigação é ativar o "Under Attack Mode" no Cloudflare e bloquear IPs de fora da América do Sul. O incidente está mitigado (médicos no Brasil conseguem logar). A resolução será a implementação de regras mais sofisticadas de WAF (Web Application Firewall).

O encerramento (cierre) do incidente envolve a desmobilização formal da War Room, o registro sumário dos eventos pelo Scribe na timeline do Jira, e o agendamento obrigatório de uma sessão de Post-Mortem em até 48 horas.

## 8. Manual Técnico Narrativo de Rollback e Procedimentos de Emergência

A capacidade de reverter alterações de infraestrutura e código em minutos é a maior arma de mitigação da Santo Pegasus. Engessar processos durante uma crise mata o paciente. Abaixo, detalhamos os procedimentos técnicos.

### 8.1 Reversão de Aplicações em Containers (AWS ECS / Fargate)

*Quando um novo deploy de um microsserviço Docker causa CrashLoopBackOff ou estouro de memória:*

- O engenheiro não deve tentar fazer um "hotfix" commitando código diretamente na branch main contornando a esteira de CI/CD.
- O procedimento de emergência consiste em forçar a AWS a utilizar a última "Task Definition" conhecida como estável.
- **Comando de Execução:** O Tech Lead executa a CLI da AWS para atualizar o serviço, ignorando temporariamente verificações de saúde estritas (Health Checks) se o Load Balancer estiver bloqueando o tráfego: ``aws ecs update-service --cluster agendio-prod --service receita-digital --task-definition receita-digital:142 --force-new-deployment``

### 8.2 Reversão de Infraestrutura (Terraform Drift)

Infraestrutura como Código (IaC) traz previsibilidade, mas um estado corrompido pode destruir ambientes inteiros.

- Antes de qualquer ``terraform apply`` em produção, a pipeline obrigatoriamente gera um artefato ``terraform plan``.
- *Se uma alteração acidentalmente remove regras de Security Groups bloqueando o tráfego entre a aplicação e o banco RDS, a mitigação é executar o plano de rollback. A edição manual via console da AWS é estritamente proibida, pois gera Drift (desalinhamento entre o código e a realidade), impossibilitando recuperações futuras.*

### 8.3 Banco de Dados: Migrações Compatíveis (Backward Compatibility)

*Modificar tabelas em bancos de dados relacionais gigantes (PostgreSQL) bloqueia linhas e gera downtime. Na Santo Pegasus, utilizando Flyway, é mandatória a política de mudanças não destrutivas em 3 fases:*

1. **Adicionar:** Criar a nova coluna sem deletar a antiga. O código escreve nas duas, mas lê da antiga.
2. **Migrar e Testar:** Os dados são copiados para a nova coluna. O código passa a ler da nova.
3. **Limpar (Semanas depois):** Apenas após a certeza absoluta de que nenhum código legado depende da estrutura antiga, a coluna velha é deletada. Comandos DDL destrutivos imediatos como ``ALTER TABLE agendamentos DROP COLUMN data_antiga;`` são bloqueados na esteira, pois um rollback da aplicação falharia por não encontrar mais a coluna que necessita.

## 9. Matemática da Confiabilidade e Gestão de Error Budgets (Orçamentos de Erro)

A confiabilidade não é um sentimento; é uma equação matemática que traduz a saúde técnica em decisões executivas de negócio.

- **Service Level Indicator (SLI):** É a métrica real e crua extraída dos servidores. Exemplo: "99,95% das requisições de geração de PDF de Receita Médica retornaram HTTP 200 em menos de 500ms".
- **Service Level Objective (SLO):** É a meta interna rigorosa que a engenharia define. Para sistemas vitais da Santo Pegasus, o SLO é 99,9% (Three Nines) medido em uma janela móvel de 30 dias.
- **Service Level Agreement (SLA):** É o contrato jurídico externo com as redes hospitalares. Fixado em 99,5%. Se a disponibilidade cair abaixo disso, a Santo Pegasus paga multas pecuniárias contratuais severas. O SLO deve ser sempre mais rígido que o SLA.

**Gestão de Error Budgets (O freio da Engenharia):** Se o SLO é 99,9%, significa que aceitamos 0,1% de falha. Em um mês (730 horas), isso equivale a um "Orçamento de Erro" de exatamente 43 minutos e 12 segundos de indisponibilidade permitida.

*Este orçamento é a moeda de troca para inovar. Cada deploy traz risco e "gasta" um pouco deste orçamento. Se uma sequência de bad deploys esgota esses 43 minutos (um burn rate excessivo), aciona-se a política mais radical da cultura SRE: o Feature Freeze (Congelamento de Lançamentos). Durante um Feature Freeze, a diretoria de produto não pode exigir o lançamento de novas funcionalidades de negócio. 100% da capacidade produtiva dos desenvolvedores é redirecionada por decreto para o pagamento de débitos técnicos, automação de testes de estresse e otimização de queries de banco de dados, até que o Orçamento de Erro seja recuperado na janela de 30 dias. A confiabilidade sempre vence a velocidade de mercado.*

## 10. Cultura 5 Porquês, O Post-Mortem e a Notificação à ANPD

Após a estabilização de um incidente SEV-1 ou SEV-2, o evento deve ser transformado em capital intelectual corporativo. O Post-Mortem é um documento público internamente, isento de apontamento de culpados, focado em entender a cadeia de falhas.

Utilizamos a técnica dos "5 Porquês" (Root Cause Analysis) para perfurar a superfície do erro:

1. Por que o sistema caiu? Porque o banco de dados principal parou de aceitar conexões.
2. Por que parou de aceitar conexões? Porque o limite máximo de Max Connections foi atingido.
3. Por que o limite foi atingido? Porque o microserviço de faturamento iniciou um loop infinito criando novas conexões sem reciclar as antigas.

4. Por que houve um loop infinito? Porque um timeout na API da adquirente de cartão de crédito não estava sendo tratado adequadamente.

5. Por que o erro de timeout chegou à produção? Porque não temos testes de caos (Chaos Testing) que simulem lentidão extrema de parceiros terceirizados na nossa esteira de CI/CD.

*A ação reparatória (Action Item) não é "dar bronca no desenvolvedor", mas sim implementar um Circuit Breaker no código e incluir injeção de falhas nos testes automatizados.*

**Conformidade Jurídica e a LGPD (Lei Geral de Proteção de Dados):** No setor de saúde (Artigo 11 da LGPD), vazamentos de dados são infrações gravíssimas. Se durante a investigação o Post-Mortem revelar que dados de prontuários, CPFs ou receitas de pacientes foram expostos indevidamente na internet (ex: um bucket S3 configurado como público), o Data Protection Officer (DPO) da Santo Pegasus assume o controle. A lei exige notificação formal e fundamentada à Autoridade Nacional de Proteção de Dados (ANPD) em um prazo estritamente de 2 (dois) dias úteis após a ciência inequívoca do evento. Ocultar o vazamento para tentar consertar silenciosamente agrava a multa (que pode chegar a R\$ 50 milhões). O Post-Mortem rigoroso serve como documento jurídico comprobatório de que a empresa agiu com diligência técnica, cooperando para reduzir possíveis ações por danos morais.

## 11. Chaos Engineering: Validando a Resiliência sob Pressão

Protocolos em papel não sobrevivem ao contato com a realidade se não forem testados. Para garantir que este manual funcione e que os sistemas de auto-recuperação operem como esperado, a Santo Pegasus pratica Engenharia do Caos (Chaos Engineering) através de eventos regulares chamados GameDays.

*Através de simulações controladas (inicialmente em ambiente de Staging, evoluindo para Produção sob monitoramento extremo), a equipe de SRE introduz falhas técnicas deliberadas no ecossistema sem avisar previamente os desenvolvedores de plantão:*

- **Caída de Zonas (AZ Failure):** Forçar o desligamento de todas as instâncias em uma Zona de Disponibilidade da AWS (ex: `sa-east-1a`) para verificar se o tráfego é automaticamente roteado para a zona `sa-east-1b` sem interrupção perceptível para o médico.
- **Injeção de Latência (Network Blackhole):** Adicionar artificialmente 5.000 milissegundos de latência nas chamadas para a API de elegibilidade da ANS para testar se os nossos Circuit Breakers abrem corretamente e ativam o fluxo de contingência (fallback) em vez de travar toda a aplicação.
- **Exaustão de Recursos:** Consumir propositalmente 100% da CPU de nós do Kubernetes para validar se o Horizontal Pod Autoscaler (HPA) cria novas réplicas de forma ágil o suficiente.

**O GameDay tem dois objetivos: testar a robustez arquitetônica do software sob estresse destrutivo e, mais criticamente, treinar a resposta humana. Ele testa se o Incident Commander consegue gerenciar o pânico, se o Tech Lead consegue encontrar os logs certos em meio ao caos e se o alerta no PagerDuty toca para a pessoa correta na hora correta. Ter "músculos treinados" na paz é o que garante a precisão cirúrgica e a proteção da vida do paciente durante a guerra de um incidente real.**

Propriedade da Santo Pegasus Soluciones. Versão 6.0 — Julho de 2026. Todos os direitos reservados. [Source: Page 6]